EXPANDING THE QUALITY–COVERAGE FRONTIER WITH COMPETITIVE FLOW MATCHING

Wenxuan Peng[1,2] **Richard Zhang**[2] **Haitian Zheng**[2] **Jaskirat Singh**[2] **Tsai-Shien Chen**[2]
Eli Shechtman[2] **Bharath Hariharan**[1] **Hadar Averbuch-Elor**[1] **Zongze Wu**[2]
[1]Cornell University [2]Adobe Research

Project page: `https://lilydaytoy.github.io/CFM/`

arXiv:2611.07055v1 [cs.CV] 4 Nov 2026

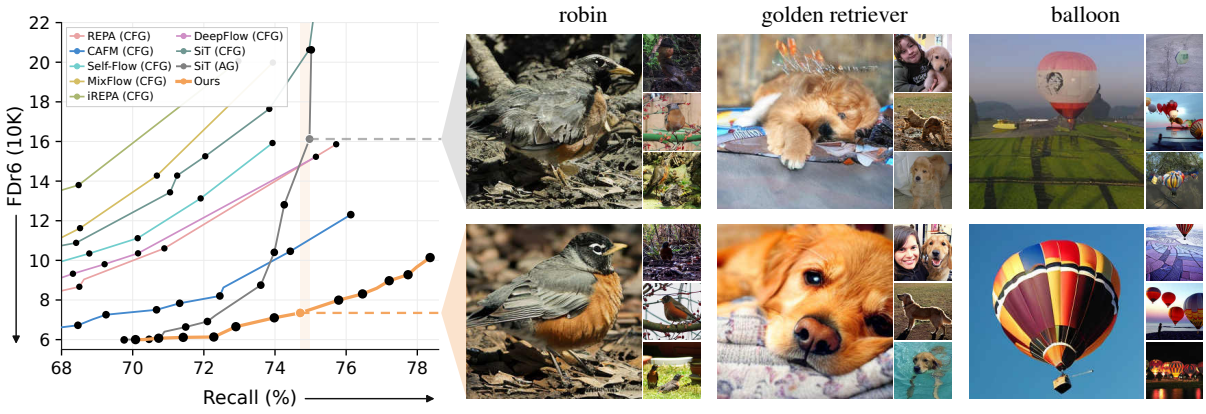
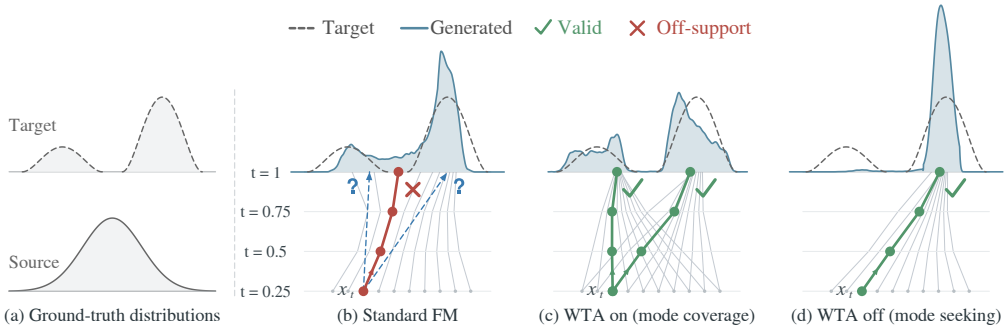


Figure 1: **Our method expands the quality–coverage frontier toward high-coverage, high-quality generation.** $\mathrm{FD}_{r,6}$–Recall Pareto frontiers reported on ImageNet-256, compare our method with recent strong flow-matching methods. Curves are obtained by sweeping each method's inference-time guidance. CFG and AG denote classifier-free guidance and AutoGuidance, respectively. Our method reaches substantially stronger performance in the high-coverage regime. This is further illustrated on the right using random samples, comparing samples obtained using the same class condition and initial noise before (top) and after (bottom) applying our approach over the SiT baseline.

ABSTRACT

Flow matching (FM) generators often face a quality-coverage tradeoff. Inference-time techniques such as classifier-free guidance raise quality, but at the cost of coverage. We hypothesize this limitation stems from a basic property of the objective – an intermediate noisy image could correspond to *many* clean images, yet the model can only predict a *single* velocity, at best averaging over the multiple possibilities. To mitigate this, we introduce competition among predictions during training. We achieve this by augmenting the predictor with a randomly drawn low-dimensional conditioning code z. For each training example, we sample K values of z to produce K candidate velocities that compete to explain the target, and apply the loss only to the best-matching candidate. This winner-take-all objective allows different modes to shape different predictions, rather than being averaged into a single compromise. Furthermore, this term naturally enables a new form of guidance, auxiliary conditioning scaling, which complements traditional guidance methods. Together, these two guidance mechanisms show that our method substantially improves the quality–coverage frontier. We validate on ImageNet-256, using two different testbeds, SiT and RAEv2. Given less than 3% of the original SiT pretraining compute for post-training, at the 67% Recall point, our method reduces $\mathrm{FD}_{r,6}$ from 19.22 to 5.91. On RAEv2, at the 70% Recall point, it reduces $\mathrm{FD}_{r,6}$ from 16.83 to 3.18.



(a) Ground-truth distributions (b) Standard FM (c) WTA on (mode coverage) (d) WTA off (mode seeking)

Figure 2: **A 1D toy illustrating conflicting velocity targets in flow matching.** (a) Transport from a unimodal source to an imbalanced bimodal target. (b) Standard FM produces off-support mass and incomplete mode coverage. (c) Winner-take-all (WTA)-trained FM sampled with auxiliary conditioning recovers both target modes while largely removing off-support mass; two fixed codes are shown to visualize distinct code-conditioned trajectories. (d) Sampling the same WTA-trained model without the auxiliary conditioning instead concentrates generation on the dominant mode.

1 Introduction

A successful generative model has two essential aspects: (1) *high-fidelity*, where each randomly drawn sample looks realistic, and (2) *broad coverage*, where the full data distribution is well-represented. Flow Matching (FM) (Lipman et al., 2023; Ma et al., 2024) and follow-ups in architecture, representations, and training objectives (Lin et al., 2026; Yu et al., 2025; Shin et al., 2025; Chefer et al., 2026) have yielded groundbreaking improvements in quality. In particular, to show impressive individual samples, inference-time techniques such as classifier-free guidance (Ho & Salimans, 2022) and improvements (Sun et al., 2024; Chung et al., 2025; Karras et al., 2024) deliberately move towards the mode of the learned distribution, often at the expense of the tails. In Fig. 1, we demonstrate that while progress has indeed been made in quality (moving down the y-axis), the *coverage* of the distribution (x-axis) has been underexplored. In this work, we demonstrate improvements in both quality and coverage, pushing the frontier significantly.

To understand where this trade-off comes from, we revisit flow matching through a 1D toy example (Fig 2). Flow matching learns a velocity field, transporting samples from a source distribution to a target distribution through a sequence of local regression problems. At training time, it constructs intermediate states by interpolating between randomly drawn source and target samples, training a predictor to the target. Importantly, a given intermediate state is compatible with *multiple* targets, each inducing a different velocity. Because the model is only equipped to predict a *single* velocity, this pulls the prediction in different directions. With finite modeling capacity, this diverts generation away from the data support and leaves valid alternatives underrepresented, as shown in Fig 2(b). This motivates our key insight: to **accommodate multiple alternatives with multiple predictions.**

To allow the model to make multiple predictions for the same noisy state, we introduce a low-dimensional Gaussian code as auxiliary conditioning. During training, we sample multiple auxiliary codes, yielding K candidate velocity predictions for the same intermediate state. Only the best-matching candidate receives the regression loss – a winner-take-all assignment explored in (Guzmán-Rivera et al., 2012; Rupprecht et al., 2017). In our Competitive Flow Matching setup, different targets can thus shape different predictions rather than being averaged into a single compromise. This enables the model to produce different velocity predictions for the same noisy state, as shown in Fig. 2(c), natively accounting for both modes. Interestingly, turning off the auxiliary code, which reverts the model to only being able to predict a single velocity, reveals an advantageous emergent property. As shown in Fig. 2(d), rather than going off-manifold, the generation is able to concentrate on the dominant target mode.

We validate this idea on class-conditional ImageNet-256 generation, systematically comparing against a broad range of methods in a shared setting. We first apply Competitive Flow Matching to the pretrained SiT (Ma et al., 2024) testbed through lightweight post-training. Without inference-time guidance, our method strengthens the pretrained generator and, by varying the auxiliary-conditioning

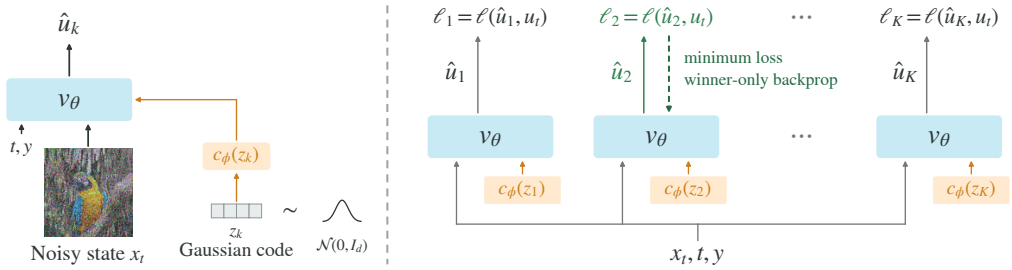
scale, exposes a smooth quality–coverage curve rather than a single operating point. More importantly, when combined with inference-time guidance, the technique changes how the generator responds to guidance: generation quality improves substantially while coverage is largely preserved, and in some cases coverage and sample variation *even both improve* with stronger guidance. Together, these effects substantially expand the quality–coverage frontier, reaching high-quality, high-coverage regions that previous methods do not access (Fig. 1). We further apply our technique to RAEv2 (Singh et al., 2026b), a state-of-the-art ImageNet generator operating in a pretrained representation space. WTA further improves this already strong model, establishing a new state of the art under its evaluation setting, with especially large gains at high coverage. Finally, we analyze how WTA reshapes the trained generator itself and uncover a complementary high-fidelity behavior induced by competition.

2 Related Work

Advances in flow matching. Flow matching has become a powerful framework for image generation. Architectural improvements have made flow-based image models increasingly scalable, while representation-based methods further strengthen the features learned during generation through external alignment, self-supervision, and intermediate supervision (Yu et al., 2025; Singh et al., 2026a; Wang et al., 2025; Jiang et al., 2026; Haghighi et al., 2026; Chefer et al., 2026; Shin et al., 2025). More recent work has also revisited the learning signal itself, for example by training on states closer to those encountered during generation (Li et al., 2026) and constructing more stable or informative velocity targets (Yang et al., 2026a; Stoica et al., 2026). Other advances improve generation through adversarial or contrastive supervision (Lin et al., 2026; Stoica et al., 2025) and cross-sample collaboration (Mo et al., 2026). These methods have pushed generation quality higher, but it is less clear whether the gains extend to broad coverage: in our controlled evaluation, broad-coverage settings remain substantially weaker than the strongest lower-coverage operating points.

Inference-time guidance. Beyond improving the generator itself, inference-time guidance is widely used to improve the fidelity of individual samples. Classifier-free guidance (CFG) (Ho & Salimans, 2022) steers generation toward the modes of the learned distribution, improving sample quality at the cost of coverage. Subsequent methods aim to keep these gains while losing less coverage: Inner CFG incorporates higher-order conditioning information (Sun et al., 2024), CFG++ attributes the degradation at strong guidance to off-manifold extrapolation and keeps the sampling update closer to the data manifold (Chung et al., 2025). AutoGuidance guides a strong model with a weaker version of itself, using their prediction difference to improve sample quality while better preserving variation (Karras et al., 2024). These methods make the quality–coverage trade-off more favorable, but their quality gains still come primarily from moving away from broad coverage. Rather than modifying the sampler, our work changes how flow matching learns from its training targets, allowing guidance to reach much higher quality at broad coverage.

Winner-take-all learning. Winner-take-all (WTA) learning handles multimodal prediction by producing several candidates per input and updating only the one closest to the target, so that candidates specialize to different modes. Multiple Choice Learning introduced this principle for structured prediction, first with an ensemble of models and later with multiple heads of a single network (Guzmán-Rivera et al., 2012; Lee et al., 2016; Rupprecht et al., 2017). IMLE applies the same nearest-candidate matching to implicit generative modeling, matching each data point to its nearest generated sample (Li & Malik, 2018), and conditional IMLE extends it to conditional generation by drawing several latent codes per input (Li et al., 2020). We bring this formulation to local velocity prediction in flow matching. For the same intermediate state, multiple auxiliary-conditioned predictions compete to match the sample-wise velocity target, allowing different target velocities to be fit by different predictions. Concurrent work on Explorative Modeling (XM) (Gladstone et al., 2026) also trains generative models with WTA, but its candidates come from different noise realizations of the same clean sample and therefore lie at different noisy states, whereas ours share a single state. Furthermore, XM targets expressivity and training efficiency, whereas we target generation quality at broad coverage.



(a) Auxiliary conditioning (b) WTA training

Figure 3: **Auxiliary conditioning and WTA training.** (a) A Gaussian code z_k is mapped through c_ϕ to condition the predictor v_θ. (b) WTA compares K predictions from one shared model against the same target. Only the best match backpropagates.

3 METHOD

Background Flow matching learns a time-dependent velocity field that transports a simple noise distribution to the data distribution (Lipman et al., 2023; Ma et al., 2024). Given a data sample $x_0 \sim p_{\text{data}}$, Gaussian noise $\epsilon \sim \mathcal{N}(0, I)$, and time t, the training procedure constructs intermediate states x_t by interpolating between data and noise points using interpolation schedule α_t, σ_t, creating sample-wise velocity targets u_t:

$$x_t = \alpha_t x_0 + \sigma_t \epsilon, \qquad u_t = \dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon. \tag{1}$$

The velocity model v_θ is trained by squared regression, and may incorporate any extra conditioning signal y,

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{x_0, \epsilon, t}\left[\|v_\theta(x_t, t, y) - u_t\|_2^2\right]. \tag{2}$$

A given state x_t and time t may correspond to many different data–noise pairs, each corresponding to a different target u_t. Standard FM fits these different targets with a single velocity prediction, which is their conditional mean:

$$v^*(x_t, t) = \mathbb{E}[u_t \mid x_t, t, y]\,. \tag{3}$$

3.1 WINNER-TAKE-ALL TRAINING

Injecting low-dimensional conditioning. To allow multiple velocity predictions for the same noisy state, we introduce a low-dimensional Gaussian code z. We use c_ϕ to map z to a backbone-compatible auxiliary conditioning signal into v_θ, as shown in Fig. 3(a). Varying z enables the network to produce different velocity predictions for the same state. Then, during training, for each noisy state x_t, we draw K independent codes, producing K candidate velocity predictions:

$$\widehat{u}_k = v_{\theta,\phi}\big(x_t, t, y, c_\phi(z_k)\big), \text{where } z_k \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, I_d),\ k \in \{1, \ldots, K\} \tag{4}$$

As shown in Figure 3(b), among the K candidates, we select the one with the smallest FM regression error and backpropagate only through the winning prediction, which minimizes

$$\mathcal{L}_{\text{WTA}}(\theta, \phi) = \mathbb{E}_{x_0, y, \epsilon, t, z_{1:K}}\left[\min_{k \in \{1,\ldots,K\}} \|\widehat{u}_k - u_t\|_2^2\right]. \tag{5}$$

Implementation details. Though this is a general objective, in our experiments, we leverage a pretrained model θ_0 and continue training the generator with this objective, which we refer to as *WTA post-training*.

The realization of the auxiliary conditioner c_ϕ depends on the backbone architecture. We use each model's native conditioning mechanism to implement c_ϕ. We take SiT as a representative example. SiT (Ma et al., 2024) is a transformer-based flow-matching generator operating in the latent space of a pretrained VAE. Its original conditioning inputs, time t and class label y, are mapped to AdaLN modulation parameters that control the scale, shift, and gating of each transformer block. To implement c_ϕ, we map the Gaussian code z through a small MLP into the same conditioning space and use a parallel AdaLN pathway to produce an auxiliary modulation of the same form. This auxiliary modulation is added to the native time–class modulation before being applied to the block. Thus, in SiT, c_ϕ corresponds to the code embedding together with the parallel AdaLN conditioning pathway. We provide the corresponding implementations for other backbones, along with exact architectural and initialization details, in the appendix.

3.2 Inference and Guidance

Once trained, there are multiple possibilities for how the auxiliary code should be used along a sampling trajectory during inference. We can either sample one code and keep it fixed throughout the generation trajectory, or independently resample the code at each sampling step. We empirically find that the former works better overall, particularly in the broad-coverage regime. Intuitively, this may be because keeping the same code may allow its influence on the predicted velocity field to remain consistent across sampling steps, while resampling may change this influence at every step, weakening the cumulative effect. Therefore, unless otherwise specified, we sample $z \sim \mathcal{N}(0, I_d)$ once and keep it fixed across all sampling steps. We provide a detailed comparison with resampled codes in the appendix.

Guidance. When combining our method with inference-time guidance, there are two adjustable axes: the auxiliary-conditioning scale and the guidance strength.

Auxiliary-conditioning scale λ. Each auxiliary code induces a different prediction. Intuitively, varying the code is like querying different code-conditioned velocity fields. Because all codes share the same network, the WTA-trained model can be viewed as a shared base predictor, with the auxiliary conditioning on top of it producing a code-dependent response. Adjusting how strongly this conditioning influences the shared predictor provides an additional inference-time control that complements conventional guidance. We realize this adjustment by multiplying $c_\phi(z)$ by a scale λ:

$$v_\lambda = v_\theta(x_t, t, y, \lambda c_\phi(z)) \,. \tag{6}$$

At $\lambda = 0$, the auxiliary conditioning is removed, leaving the post-trained base model.

Guidance strength γ. The auxiliary-conditioning control can be combined with inference-time guidance. A common form of guidance extrapolates the generator prediction away from a guiding prediction v_G, which may come, for example, from an unconditional prediction (classifier-free guidance Ho & Salimans (2022)) or a weaker model (autoguidance Karras et al. (2024)). The guided prediction is then

$$v_{\lambda,\gamma} = v_\lambda + \gamma\,(v_\lambda - v_G) \,. \tag{7}$$

4 Experiments

We evaluate our method primarily on class-conditional ImageNet-256 generation. We first use SiT as a controlled testbed to evaluate how our post-training affects generation quality and coverage. We then test whether these improvements generalize beyond SiT using RAEv2, and analyze the role of competition.

4.1 SiT on ImageNet

Setup. We use SiT as our main testbed because many recent flow-matching methods build on it, allowing controlled comparisons within a common model family.

Our main model starts from the official pretrained SiT-XL/2 checkpoint and is post-trained with the WTA objective for 10 epochs with $K = 8$. For guidance, we instantiate Eq. 7 with AutoGuidance (AG) (Karras et al., 2024): the guiding prediction v_G comes from a smaller SiT-B/2, also post-trained with our method using $K = 8$ and run auxiliary-off ($\lambda = 0$). The pretrained SiT baseline uses the same AG formulation, with pretrained SiT-XL/2 and SiT-B/2 as the generator and guiding model.

To separate differences in the learned generators from differences in sampling heuristics, the controlled SiT comparisons, including Table 1, use a 50-step Euler ODE solver with uniformly spaced timesteps. For the methods we compare with, we re-evaluate them under this recipe when their original setup differs, retaining the guidance formulation used in their original papers and sweeping its strength.

We report $\mathrm{FD}_{r,6}$ (Yang et al., 2026b) as our primary metric; it aggregates Fréchet distances across six representation spaces, including the Inception space underlying FID. We also report Precision (sample fidelity) and Recall (coverage of the data distribution), with Inception Score (IS) as a supporting quality metric and within-class DreamSim (Fu et al., 2023) distance as a supporting diversity metric. Operating-point curves use 10K generated samples; key numerical comparisons use 50K.

Varying the auxiliary conditioning scale λ. We first examine the effect of auxiliary conditioning by varying λ in the post-trained generator while fixing $\gamma = 0$. Starting from the pretrained SiT checkpoint, 10 epochs of our post-training yield clear gains, whereas continuing standard FM training does not improve generation.

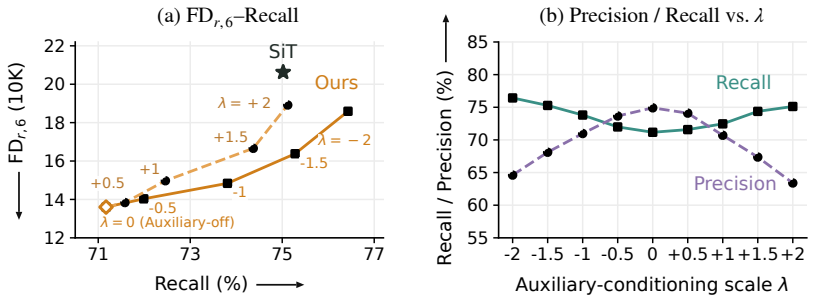


Figure 4: **Our method improves generation and enables smooth operating-point control.** (a) At $\gamma = 0$, varying the auxiliary-conditioning scale λ lowers $\mathrm{FD}_{r,6}$ relative to pretrained SiT, including at matched or higher Recall. (b) Increasing $|\lambda|$ from auxiliary-off ($\lambda = 0$) trades Precision for Recall.

As shown in Fig. 4(a), varying λ yields operating points with lower $\mathrm{FD}_{r,6}$ than pretrained SiT, including at matched and even higher Recall. The auxiliary-conditioning scale λ also provides a smooth inference-time knob for controlling the generator's operating point. As shown in Fig. 4(b), the auxiliary-off setting ($\lambda = 0$) lies at high Precision, and increasing $|\lambda|$ moves the generator steadily toward higher Recall. Interestingly, we empirically observe similar overall trends for both signs of λ, with negative values generally yielding lower $\mathrm{FD}_{r,6}$. We therefore present the negative-λ branch in the guided frontier below and compare both signs in Appendix B.3.

Under the standard 50K-sample evaluation at $\gamma = 0$, at matched Recall (66.91% for pretrained SiT and 66.83% for our model), we reduce $\mathrm{FD}_{r,6}$ from 19.22 to 14.94 and FID from 11.17 to 7.22. The auxiliary-off setting ($\lambda = 0$) reaches an even lower $\mathrm{FD}_{r,6}$ of 12.15 (FID 4.59). *Our method produces a stronger generator whose quality–coverage trade-off can be smoothly controlled through λ alone.*

Combining with Guidance. We next apply guidance on top of these operating points: for each fixed λ, we sweep the guidance strength γ to trace a guided trajectory. Figure 5(a) shows that pretrained SiT and our model respond very differently to guidance. For pretrained SiT, increasing guidance lowers $\mathrm{FD}_{r,6}$ but steadily reduces Recall. For our model, at a fixed λ, stronger guidance

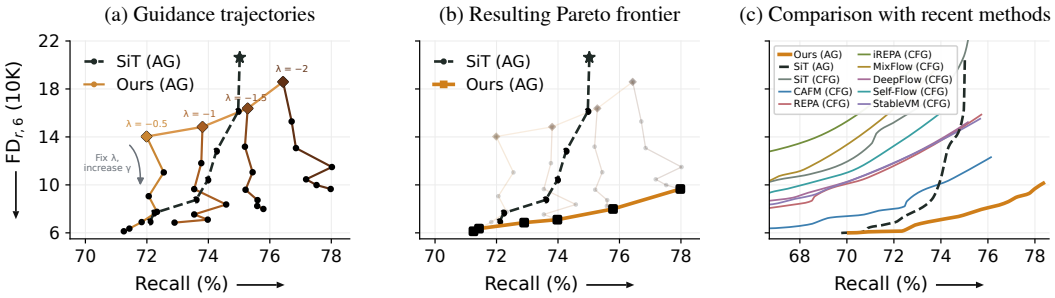


Figure 5: **Guidance expands our model's Pareto frontier toward broad coverage.** (a) AutoGuidance trajectories for pretrained SiT and our post-trained model in $\mathrm{FD}_{r,6}$–Recall space. For each fixed λ, we sweep $\gamma = 0, 0.25, \ldots, 1.5$, with successive points ordered from top to bottom. (b) The resulting Pareto frontier obtained by combining trajectories across λ, compared with SiT. (c) Comparison with recent advances in flow matching over their respective guidance sweeps.

moves the trajectory mostly downward: $\mathrm{FD}_{r,6}$ decreases substantially with little or no Recall loss, while at high-coverage operating points Recall can even increase.

Combining the guided trajectories across λ gives the Pareto frontier in Fig. 5(b), which extends beyond guided SiT toward both lower $\mathrm{FD}_{r,6}$ and higher Recall. Figure 5(c) compares this frontier with recent flow-matching methods over their respective guidance sweeps: our method performs best overall, with its largest advantage in the broad-coverage regime.

We further analyze this synergy with guidance, particularly in the high-coverage regime. In addition to Recall, we measure within-class diversity using mean pairwise DreamSim (Fu et al., 2023) distance. Echoing the Recall trends above, stronger guidance reduces diversity for pretrained SiT but steadily increases it for our model at high-coverage settings (Fig. 6).

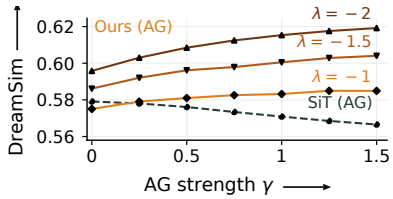


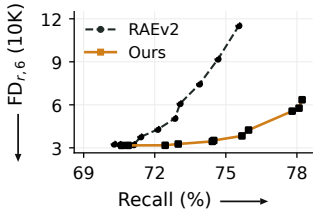
Figure 6: DreamSim distance as AG strength γ increases: SiT vs. Ours.

Table 1: **Guided comparison under the standard 50K-sample evaluation.** (a) Performance at each method's lowest $\mathrm{FD}_{r,6}$. (b) Lowest $\mathrm{FD}_{r,6}$ at Recall $\approx 67\%$; light-gray Recall values mark methods whose highest measured Recall is below 67%. The six raw Fréchet distances (including ADM FID) appear in smaller gray type; $\mathrm{FD}_{r,6}$ averages their normalized values.

	(a) Performance @ best $\mathrm{FD}_{r,6}$										**(b) Performance @ Recall $\approx 67\%$**									
	Multi-encoder Fréchet distances ↓										Multi-encoder Fréchet distances ↓									
Method	FID (ADM)	ConvNeXt	DINOv2	MAE	SigLIP	CLIP	$\mathrm{FD}_{r,6}$↓	Recall (%) ↑	Precision (%) ↑	IS ↑	FID (ADM)	ConvNeXt	DINOv2	MAE	SigLIP	CLIP	$\mathrm{FD}_{r,6}$↓	Recall (%) ↑	Precision (%) ↑	IS ↑
CAFM	3.30	85.58	51.49	0.13	**5.06**	58.31	4.86	57.1	83.8	342.7	3.84	225.73	135.82	0.24	10.91	106.67	9.84	67.3	72.1	181.9
REPA	4.79	132.70	52.50	0.19	5.69	60.93	5.65	49.7	88.0	388.8	6.42	302.84	171.30	0.37	14.43	136.53	13.13	67.2	69.7	162.5
MixFlow†	6.52	156.09	65.02	0.20	6.40	67.52	6.49	46.9	88.6	396.1	9.35	495.73	252.08	0.50	21.91	169.85	18.57	66.0	68.0	133.5
Self-Flow†	6.37	147.99	55.50	0.18	6.05	60.74	5.94	46.5	89.8	388.1	7.63	400.46	197.52	0.35	16.29	143.51	14.51	66.1	69.4	138.1
StableVM	5.47	140.84	52.32	0.18	5.75	60.29	5.72	48.0	88.6	391.1	7.45	360.07	188.04	0.38	15.46	142.35	14.14	67.9	68.7	147.3
DeepFlow	5.64	143.20	52.50	0.19	5.84	60.50	5.83	48.2	88.8	**398.6**	7.16	359.93	186.26	0.38	15.47	139.92	14.00	67.3	68.7	148.2
iREPA†	7.26	170.67	73.12	0.28	9.50	71.03	7.99	40.5	**90.8**	373.4	9.81	551.60	256.33	0.49	22.97	160.22	18.81	65.2	67.9	117.6
SiT (CFG)†	6.52	150.02	62.21	0.20	6.53	64.86	6.40	46.6	89.2	383.7	11.17	572.67	275.86	0.47	21.39	177.22	19.22	66.9	66.3	118.6
SiT (AG)†	**2.13**	85.92	50.46	**0.11**	5.26	52.29	4.53	**62.2**	78.1	236.9	11.17	572.67	275.86	0.47	21.39	177.22	19.22	66.9	66.3	118.6
Ours	2.29	**77.44**	**46.13**	0.13	5.24	**49.06**	**4.44**	60.4	79.9	258.2	**2.20**	**122.93**	**69.43**	**0.20**	**6.72**	**61.19**	**5.91**	67.4	**73.0**	**210.5**

Finally, Table 1 reports the full set of metrics under the standard 50K-sample evaluation across methods. At the best overall operating point, our method achieves the lowest $\mathrm{FD}_{r,6}$ in the comparison, reaching 4.44 (Table 1a). The gap widens in the broad-coverage regime (Recall $\approx 67\%$, Table 1b). Competing methods degrade sharply here, with higher $\mathrm{FD}_{r,6}$ and lower Precision and Inception Score. Our model instead stays near its best, reaching $\mathrm{FD}_{r,6} = 5.91$ with the highest Precision and Inception Score among the compared methods. *With our method, stronger guidance improves quality while largely preserving coverage, extending the quality–coverage frontier toward broad coverage.*

4.2 Extending to RAEv2



Method	Performance @ best $\mathrm{FD}_{r,6}$				Performance @ Recall $\approx 70\%$			
	$\mathrm{FD}_{r,6}\downarrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	IS $\uparrow$	$\mathrm{FD}_{r,6}\downarrow$	Recall (%) $\uparrow$	Precision (%) $\uparrow$	IS $\uparrow$
RAEv2	2.07	64.72	**79.39**	261.68	16.83	69.09	52.39	111.47
Ours	**2.02**	**65.71**	78.30	**265.82**	**3.18**	70.32	**71.25**	**182.39**

Figure 7: **Our method further improves RAEv2.** Left: $\mathrm{FD}_{r,6}$–Recall frontiers (10K samples): our method extends beyond pretrained RAEv2, most at broad coverage. Right: On the 50K-sample evaluation, our method improves the best overall $\mathrm{FD}_{r,6}$ with a larger gain near 70% Recall.

We test generalization on a second testbed, RAEv2 (Singh et al., 2026b). Whereas SiT operates in a VAE latent space, RAEv2 generates in the representation space of a pretrained visual encoder and achieves state-of-the-art ImageNet generation. RAEv2 has its own guidance mechanism, which we call RAEv2-guidance: in the notation of Sec. 3.2, the guiding prediction v_G comes from an intermediate head within the same forward pass. For our model, both predictions therefore share the same λ. We post-train the pretrained RAEv2 generator for 10 epochs with $K = 8$ competing predictions. Both the pretrained and post-trained models use RAEv2's native inference recipe, including its timestep shifting and guidance window. We sweep the RAEv2-guidance strength for each, and additionally vary λ for our model.

As shown on the left of Fig. 7, our method substantially expands the $\mathrm{FD}_{r,6}$–Recall frontier beyond pretrained RAEv2, reaching lower $\mathrm{FD}_{r,6}$ and extending to broad-coverage operating points. At the best overall point, $\mathrm{FD}_{r,6}$ improves from 2.07 to 2.02. The gap is much larger near 70% Recall, where our model reaches $\mathrm{FD}_{r,6} = 3.18$ at 70.32% Recall, versus 16.83 at 69.09% for RAEv2. At these operating points, our method also raises Precision from 52.39% to 71.25% and Inception Score from 111.47 to 182.39, as shown on the right of Fig. 7. *Our method generalizes across testbeds, improving even the already strong RAEv2 generator, with the largest gains at broad coverage.*

4.3 Analyzing the Role of Competition

We next analyze how our post-training reshapes the generator and what this reveals about the role of competition. We start with the base model itself, evaluated by removing the auxiliary conditioning ($\lambda = 0$), and compare it with its pretrained counterpart across generator families. We then use SiT to study how the base model changes as competition scales with K, and finally extend this analysis across λ.

Removing the auxiliary conditioning. Using $\lambda = 0$ removes the auxiliary conditioning $c_\phi(z)$ and the ability to predict multiple velocities, leaving a fine-tuned version of the original base model. How does this base model compare to its pretrained version? Table 2 evaluates this generator across representative generator families, including VAE-latent SiT (Ma et al., 2024), pixel-space JiT (Li & He, 2025), and representation-autoencoder generators RAE (Zheng et al., 2026) and RAEv2 (Singh et al., 2026b). Across all four models, the base model after our post-training achieves lower $\mathrm{FD}_{r,6}$ and higher Inception Score, and shifts toward a higher-Precision, lower-Recall regime. Interestingly, *competitive post-training improves the base model itself and shapes it into a higher-fidelity generator.*

Table 2: **Base model evaluation.** For each generator, we compare the pretrained model with its base model after our post-training ($K = 8$), evaluated at $\lambda = 0$. Each model is evaluated following the inference and evaluation protocol of its original work (Appendix A.1.7).

	SiT		JiT		RAE		RAEv2	
Metric	Pre-trained	Ours	Pre-trained	Ours	Pre-trained	Ours	Pre-trained	Ours
$\text{FD}_{r,6} \downarrow$	15.76	**10.59**	15.17	**14.15**	4.41	**3.64**	3.89	**3.52**
Recall (%) $\uparrow$	**67.65**	62.83	**66.46**	62.03	**63.80**	58.73	**62.08**	55.49
Precision (%) $\uparrow$	68.71	**75.70**	68.06	**72.05**	79.13	**83.53**	79.84	**85.76**
Inception Score $\uparrow$	131.86	**178.47**	151.49	**168.01**	240.36	**269.29**	226.72	**278.71**

Scaling competition. The effect on the base model is closely tied to competition. As shown in Fig. 8(a), when $K = 1$, no competition occurs, and post-training leaves the base model nearly unchanged from pretrained SiT. As more predictions compete for each target, the base model shifts systematically toward higher Precision and lower $\text{FD}_{r,6}$.This effect on $\text{FD}_{r,6}$ is more pronounced with shorter sampling trajectories (Fig. 8(b)): as K increases, the $\text{FD}_{r,6}$ of 8- and 4-step generation continues to drop substantially, narrowing the gap to 50-step generation.

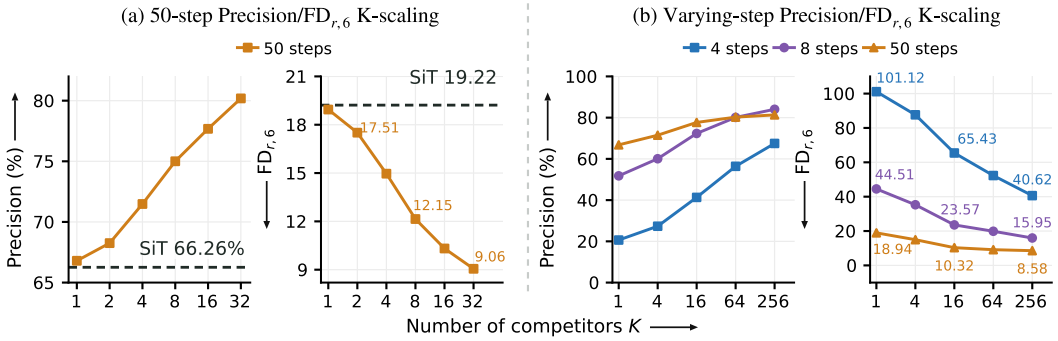


Figure 8: **Stronger competition increasingly reshapes the base model.** (a) Under 50-step sampling, increasing the number of competitors K steadily raises Precision while lowering $\text{FD}_{r,6}$. (b) The same trend is amplified under shorter sampling trajectories: 4- and 8-step generation continues to improve at larger K, progressively narrowing the gap to the 50-step reference.

This effect is not confined to the base model at $\lambda = 0$. Figure 9 shows how Precision and Recall change with the auxiliary-conditioning scale λ for various K values. With $K = 1$, both metrics remain nearly flat as λ varies. As K increases, the response to λ becomes progressively stronger: moving λ away from zero shifts the same trained network from the higher-Precision base model toward higher Recall.

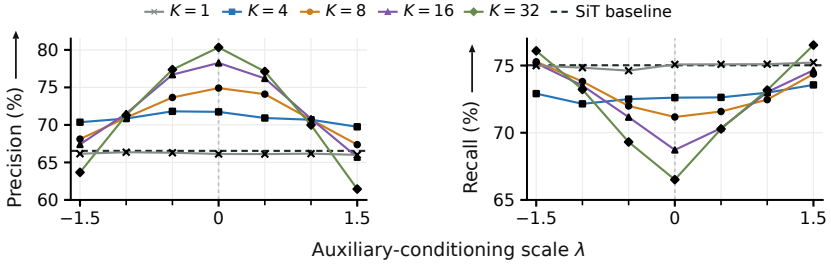


Figure 9: **Competition reshapes the generator across the full range of auxiliary-conditioning scales.** Precision (left) and Recall (right) as the auxiliary-conditioning scale λ varies for models post-trained with different numbers of competing predictions K. The dashed horizontal line denotes pretrained SiT baseline. All results use $\gamma = 0$.

5 CONCLUSION

A standard flow-matching model is trained toward a single operating point, leaving movement along the quality–coverage trade-off largely to inference-time guidance. In this work, we have shown that with WTA post-training, a single model instead holds two complementary behaviors: its base model operates at high fidelity, while auxiliary conditioning moves it toward high coverage. The auxiliary-conditioning scale λ provides smooth control over this quality–coverage trade-off and can be used on its own or combined with existing guidance methods. Most notably, competition also changes how the generator responds to guidance. For pretrained models, stronger guidance improves quality at the cost of coverage; for WTA-post-trained models, it improves quality while largely preserving coverage, and at high-coverage operating points it can even increase Recall and within-class diversity.

A practical limitation of our method is its additional training cost: while it does not require training models from scratch, selecting the winner involves evaluating K candidate predictions for each training state, and this cost grows with K. Looking ahead, the auxiliary codes themselves merit closer study. Although WTA training only asks predictions to compete for each training target, without requiring any code to play a consistent role across states or sampling trajectories, a fixed code works well at inference: strengthening the auxiliary conditioning progressively increases coverage, and this combines well with guidance. The auxiliary-conditioned predictions thus appear to learn more structure than the objective explicitly asks for. Understanding this structure better, and learning or using auxiliary conditions in more adaptive ways, could further expand the quality–coverage frontier.

REFERENCES

Hila Chefer, Patrick Esser, Dominik Lorenz, Dustin Podell, Vikash Raja, Vinh Tong, Antonio Torralba, and Robin Rombach. Self-supervised flow matching for scalable multi-modal synthesis. *arXiv preprint arXiv:2603.06507*, 2026.

Hyungjin Chung, Jeongsol Kim, Geon Yeong Park, Hyelin Nam, and Jong Chul Ye. Cfg++: Manifold-constrained classifier free guidance for diffusion models. In *International Conference on Learning Representations*, volume 2025, pp. 30824–30850, 2025.

Prafulla Dhariwal and Alexander Nichol. Diffusion models beat GANs on image synthesis. In *Advances in Neural Information Processing Systems*, 2021.

Stephanie Fu, Netanel Tamir, Shobhita Sundaram, Lucy Chai, Richard Zhang, Tali Dekel, and Phillip Isola. DreamSim: Learning new dimensions of human visual similarity using synthetic data. In *Advances in Neural Information Processing Systems*, volume 36, pp. 50742–50768, 2023.

Alexi Gladstone, Heng Ji, and Yilun Du. Explorative modeling: Unlocking a third pretraining axis and end-to-end generation. *arXiv preprint arXiv:2607.27372*, 2026.

Abner Guzmán-Rivera, Dhruv Batra, and Pushmeet Kohli. Multiple choice learning: Learning to produce multiple structured outputs. *Advances in Neural Information Processing Systems*, 2012.

Yasaman Haghighi, Bastien van Delft, Mariam Hassan, and Alexandre Alahi. Layersync: Self-aligning intermediate layers. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 29918–29950, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/32eb183794ef5ef9a3ab1d40a3d2b303-Paper-Conference.pdf.

Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022.

Dengyang Jiang, Mengmeng Wang, Liuzhuozheng Li, Lei Zhang, Haoyu Wang, Wei Wei, Guang Dai, Yanning Zhang, and Jingdong Wang. Representation alignment for diffusion transformers without external components. In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 18884–18915, 2026. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/201e691e8a0940291dc591eb0439a789-Paper-Conference.pdf.

Tero Karras, Miika Aittala, Tuomas Kynkäänniemi, Jaakko Lehtinen, Timo Aila, and Samuli Laine. Guiding a diffusion model with a bad version of itself. In *Advances in Neural Information Processing Systems*, 2024.

Tuomas Kynkäänniemi, Tero Karras, Samuli Laine, Jaakko Lehtinen, and Timo Aila. Improved precision and recall metric for assessing generative models. *Advances in neural information processing systems*, 32, 2019.

Stefan Lee, Senthil Purushwalkam, Michael Cogswell, Viresh Ranjan, David Crandall, and Dhruv Batra. Stochastic multiple choice learning for training diverse deep ensembles. *Advances in Neural Information Processing Systems*, 2016.

Hui Li, Jiayue Lyu, Fu-Yun Wang, Kaihui Cheng, Siyu Zhu, and Jingdong Wang. Mixflow training: Alleviating exposure bias with slowed interpolation mixture. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9095–9105, 2026.

Ke Li and Jitendra Malik. Implicit maximum likelihood estimation. *arXiv preprint arXiv:1809.09087*, 2018.

Ke Li, Shichong Peng, Tianhao Zhang, and Jitendra Malik. Multimodal image synthesis with conditional implicit maximum likelihood estimation. *International Journal of Computer Vision*, 128(10):2607–2628, 2020.

Tianhong Li and Kaiming He. Back to basics: Let denoising generative models denoise. *arXiv preprint arXiv:2511.13720*, 2025.

Shanchuan Lin, Ceyuan Yang, Zhijie Lin, Hao Chen, and Haoqi Fan. Continuous adversarial flow models. In *European Conference on Computer Vision*, pp. 645–663. Springer, 2026.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *International Conference on Learning Representations*, 2023.

Nanye Ma, Mark Goldstein, Michael S. Albergo, Nicholas M. Boffi, Eric Vanden-Eijnden, and Saining Xie. Sit: Exploring flow and diffusion-based generative models with scalable interpolant transformers. In *European Conference on Computer Vision*, 2024.

Sicheng Mo, Thao Nguyen, Richard Zhang, Nick Kolkin, Siddharth Srinivasan Iyer, Eli Shechtman, Krishna Kumar Singh, Yong Jae Lee, Bolei Zhou, and Yuheng Li. Group diffusion: Enhancing image generation by unlocking cross-sample collaboration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 35702–35712, 2026.

William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2023.

Christian Rupprecht, Iro Laina, Robert DiPietro, Maximilian Baust, Federico Tombari, Nassir Navab, and Gregory D. Hager. Learning in an uncertain world: Representing ambiguity through multiple hypotheses. In *Proceedings of the IEEE International Conference on Computer Vision*, 2017.

Inkyu Shin, Chenglin Yang, and Liang-Chieh Chen. Deeply supervised flow-based generative models. In *2025 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 16535–16544. IEEE, 2025.

Jaskirat Singh, Xingjian Leng, Zongze Wu, Liang Zheng, Richard Zhang, Eli Shechtman, and Saining Xie. What matters for representation alignment: Global information or spatial structure? In C. Vondrick, B. Hariharan, C. Raffel, L. Pinto, D. Yang, and A. Faust (eds.), *International Conference on Learning Representations*, volume 2026, pp. 33807–33845, 2026a. URL https://proceedings.iclr.cc/paper_files/paper/2026/file/3929a7785bd56f57edcff0152ab41289-Paper-Conference.pdf.

Jaskirat Singh, Boyang Zheng, Zongze Wu, Richard Zhang, Eli Shechtman, and Saining Xie. Improved baselines with representation autoencoders. *arXiv preprint arXiv:2605.18324*, 2026b.

George Stoica, Vivek Ramanujan, Xiang Fan, Ali Farhadi, Ranjay Krishna, and Judy Hoffman. Contrastive flow matching. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 1185–1194, October 2025.

George Stoica, Sayak Paul, Matthew Wallingford, Vivek Ramanujan, Abhay Nori, Winson Han, Ali Farhadi, Ranjay Krishna, and Judy Hoffman. Posterior augmented flow matching. In *European Conference on Computer Vision*, 2026.

Shikun Sun, Longhui Wei, Zhicai Wang, Zixuan Wang, Junliang Xing, Jia Jia, and Qi Tian. Inner classifier-free guidance and its taylor expansion for diffusion models. In *International Conference on Learning Representations*, volume 2024, pp. 4360–4376, 2024.

Ziqiao Wang, Wangbo Zhao, Yuhao Zhou, Zekai Li, Zhiyuan Liang, Mingjia Shi, Xuanlei Zhao, Pengfei Zhou, Kaipeng Zhang, Zhangyang Wang, Kai Wang, and Yang You. Repa works until it doesn't: Early-stopped, holistic alignment supercharges diffusion training, 2025. URL https://arxiv.org/abs/2505.16792.

Donglin Yang, Yongxing Zhang, Xin Yu, Liang Hou, Xin Tao, Pengfei Wan, Xiaojuan Qi, and Renjie Liao. Stable velocity: A variance perspective on flow matching. In *International Conference on Machine Learning*, 2026a.

Jiawei Yang, Zhengyang Geng, Xuan Ju, Yonglong Tian, and Yue Wang. Representation fréchet loss for visual generation. *arXiv preprint arXiv:2604.28190*, 2026b.

Sihyun Yu, Sangkyung Kwak, Huiwon Jang, Jongheon Jeong, Jonathan Huang, Jinwoo Shin, and Saining Xie. Representation alignment for generation: Training diffusion transformers is easier than you think. In Y. Yue, A. Garg, N. Peng, F. Sha, and R. Yu (eds.), *International Conference on Learning Representations*, volume 2025, pp. 87400–87442, 2025. URL https://proceedings.iclr.cc/paper_files/paper/2025/file/d9e42b4d7163931f3689d6d6fbaa11d0-Paper-Conference.pdf.

Boyang Zheng, Nanye Ma, Shengbang Tong, and Saining Xie. Diffusion transformers with representation autoencoders. In *International Conference on Learning Representations*, volume 2026, pp. 35791–35820, 2026.

SUPPLEMENTARY MATERIAL

This supplement provides implementation and evaluation details, additional quantitative results, inference ablations, analyses of generated variation, and computational costs.

CONTENTS

A Implementation and Evaluation Protocols **15**

A.1 Backbone-Specific WTA Implementations 15

A.2 Implementation Details of the One-Dimensional Toy 18

A.3 Guidance Search and Evaluation 19

B Additional Evaluation and Ablations **21**

B.1 Precision–Recall Evaluation 21

B.2 Additional Model Capacity and Continued Training 21

B.3 Signs of Auxiliary Conditioning 23

B.4 Effect of Competition Level K 23

B.5 Auxiliary-Code Sampling Policies 25

B.6 Reusable Auxiliary-Code Signatures 26

B.7 Guidance Configurations 27

C Qualitative Results **29**

C.1 Qualitative Improvements in the Base Model 29

C.2 Structured Variation Under Auxiliary Conditioning and Guidance 31

D Computational Cost **39**

D.1 Training Cost and Scaling with the Number of Candidates 39

A Implementation and Evaluation Protocols

A.1 Backbone-Specific WTA Implementations

We implement WTA by augmenting each pretrained generator with an auxiliary code-conditioning pathway. This section specifies the architecture, initialization, native regression loss, and post-training configuration for SiT, JiT, RAE, and RAEv2.

A.1.1 Shared training construction

We use the notation of Eqs. (4)–(5): x_0 denotes a clean training sample in the generator's input space, ϵ is the Gaussian noise used to construct x_t, and $z \sim \mathcal{N}(0, I_d)$ is an independent auxiliary code, with $d = 8$. The pretrained backbone already conditions on the timestep t and class label y, represented by embeddings $e_t(t)$ and $e_y(y)$. Depending on the backbone, these embeddings provide conditioning through its existing modulation modules, through in-context tokens processed together with image tokens, or through both.

We denote this complete auxiliary-conditioning pathway by c_ϕ. The pathway first maps the auxiliary code to an embedding through E_ϕ. This additional conditioning embedding is then incorporated through the backbone's native conditioning mechanism. For modulation-based injection, the auxiliary contribution is combined with the native conditioning along the modulation pathway. For context-token injection, the auxiliary embedding is added to the corresponding native context tokens. The backbone-specific descriptions below specify the exact injection points. All added parameters are collectively denoted by ϕ. We load the pretrained generator weights and initialize the added pathway to contribute zero to the forward computation. Both the shared generator parameters θ and auxiliary parameters ϕ are updated by this objective.

A.1.2 SiT

Representation and native conditioning. SiT-XL/2 (Ma et al., 2024) operates on $4 \times 32 \times 32$ VAE latents for 256×256 images. We retain the pretrained VAE, sample its posterior during training, and apply the native latent scale 0.18215. The generator has 28 Transformer blocks of width $H = 1152$, 16 attention heads, and latent patch size 2. Each Transformer block uses native AdaLN modulation to condition on $e_t(t) + e_y(y)$. Its modulation mapping $M^{(\ell)}_{\text{native}}$, implemented as SiLU followed by a linear layer, produces $6H$ values: a shift, scale, and residual gate for each of the attention and MLP sublayers. Shifts and scales modulate LayerNorm outputs. Residual gates scale the sublayer outputs. These vectors are broadcast over spatial tokens.

Auxiliary conditioning. The code embedding E_ϕ is a two-layer SiLU MLP shared across blocks, with dimensions $8 \to 1152 \to 1152$:

$$E_\phi(z) = W_2 \,\mathrm{SiLU}(W_1 z + b_1) + b_2. \tag{8}$$

Following the native AdaLN modulation structure, we introduce an additional auxiliary modulation mapping $M^{(\ell)}_\phi : \mathbb{R}^H \to \mathbb{R}^{6H}$ in each block, also implemented as SiLU followed by a linear layer. It maps the auxiliary embedding to additional shifts, scales, and residual gates, which are added to their native counterparts:

$$M^{(\ell)}_{\text{native}}(e_t(t) + e_y(y)) + c^{(\ell)}_\phi(z), \qquad c^{(\ell)}_\phi(z) = M^{(\ell)}_\phi(E_\phi(z))\,. \tag{9}$$

The final output layer similarly receives an added $2H$-dimensional shift and scale from its own SiLU–Linear projection of $E_\phi(z)$.

At initialization, the first embedding weight W_1 has independent Gaussian entries with standard deviation 0.02. The parameters b_1, W_2, and b_2 are zero. All added modulation projections use Xavier-uniform weights and zero biases. Thus the auxiliary contribution is initially zero, while the nonzero projection weights provide a gradient path to the embedding's final layer.

A.1.3 JiT

Representation and native conditioning. JiT-H/16 (Li & He, 2025) predicts clean images in pixel space. It has 32 blocks of width $H = 1280$, 16 attention heads, 16×16 image patches, and a patch-embedding bottleneck of width 256. Each Transformer block uses native AdaLN modulation to condition on $e_t(t) + e_y(y)$. JiT also uses native in-context conditioning through 32 class-context tokens: with native context-position embeddings p_j, these are $h_j^{\mathrm{ctx}} = e_y(y) + p_j$ for $j = 1, \ldots, 32$. They are concatenated with the image tokens immediately before the eleventh Transformer block and participate in subsequent self-attention.

Auxiliary conditioning. We construct the auxiliary embedding $E_\phi(z)$ using a two-layer SiLU MLP with dimensions $8 \to 1280 \to 1280$, and set

$$c_\phi(z) = E_\phi(z). \tag{10}$$

Following these two native conditioning mechanisms, we add the same auxiliary embedding to the input of the native AdaLN modulation modules and to every class-context token:

$$e_t(t) + e_y(y) + c_\phi(z), \qquad h_j^{\mathrm{ctx}} \leftarrow h_j^{\mathrm{ctx}} + c_\phi(z). \tag{11}$$

The JiT model in Table 2 uses a variant that additionally adds this same auxiliary embedding to the image patch tokens before the Transformer blocks.

The auxiliary pathway is initialized to contribute zero, preserving the pretrained predictor at the start of post-training.

A.1.4 RAE

Representation and native conditioning. We retain the pretrained representation autoencoder (Zheng et al., 2026), using a DINOv2-with-registers Base encoder with 224×224 encoder input, a ViT-XL image decoder, and the native latent normalization statistics. The stage-two generator operates on $768 \times 16 \times 16$ latents. Its DDT consists of 28 backbone blocks of width $H_b = 1152$ and two wide-head blocks of width $H_h = 2048$, with patch size 1 and 16 attention heads in each stack. These stacks belong to the generative Transformer. The stage-one image encoder and decoder remain frozen. Both stacks use native RMSNorm-based modulation. Their modulation mappings, implemented as SiLU followed by a linear layer, produce shifts, scales, and residual gates for the attention and MLP sublayers. Shifts and scales modulate RMSNorm outputs. Residual gates scale the sublayer outputs. The backbone modulation input is $\mathrm{SiLU}(e_t(t) + e_y(y))$. For an encoded image-token feature h_i, the wide-head modulation input is the native projection of $\mathrm{SiLU}(e_t(t) + h_i)$ to width H_h. This input therefore also depends on the encoded image features.

Auxiliary conditioning. Two independent two-layer SiLU MLPs embed the same code: $E_\phi^b : 8 \to 1152 \to 1152$ and $E_\phi^h : 8 \to 2048 \to 2048$. Following the native modulation structure, we introduce an auxiliary modulation mapping $M_\phi^{(\ell)} : \mathbb{R}^{H_s} \to \mathbb{R}^{6H_s}$ for each block in stack $s \in \{b, h\}$. Implemented as SiLU followed by a linear layer, it produces additional shifts, scales, and residual gates:

$$c_\phi^{(\ell)}(z) = M_\phi^{(\ell)}\big(E_\phi^s(z)\big), \tag{12}$$

which is added to that block's native modulation output. The final layer receives an auxiliary shift and scale from a separate $2H_h$-dimensional SiLU–Linear projection of $E_\phi^h(z)$. Both embeddings and all added projections follow the SiT initialization: Gaussian first-layer embedding weights with standard deviation 0.02, zero final-layer embedding weights, zero embedding biases, and Xavier-uniform projection weights with zero biases.

A.1.5 RAEv2

Representation and native conditioning. RAEv2 uses the pretrained DINOv3-L representation with features from layers $11, 13, 15, 17, 19, 21, 23$, a ViT-XL image decoder, and the native normal-

ization statistics. The stage-two input has shape $1024 \times 16 \times 16$. The generator has 28 backbone blocks of width 1440 with 20 attention heads, followed by two wide-head blocks of width 2048 with 16 heads. The backbone uses native in-context conditioning: it processes image tokens together with four native timestep tokens and eight native class-context tokens, denoted by h_j^{ctx}.

Auxiliary conditioning. The code embedding E_ϕ maps z to additive conditioning for the eight native class-context tokens, each of width 1440. Here $c_\phi = E_\phi$, and $[c_\phi(z)]_j$ denotes the auxiliary contribution at context position j. Following this native context-token mechanism, we add each contribution to the corresponding native class-context token:

$$h_j^{\mathrm{ctx}} \leftarrow h_j^{\mathrm{ctx}} + [c_\phi(z)]_j, \qquad j = 1, \ldots, 8. \tag{13}$$

The auxiliary pathway is initialized to contribute zero, preserving the pretrained predictor at the start of post-training.

A.1.6 Native losses and post-training settings

We post-train all four generators on the ImageNet-1K training set at 256×256 image resolution. We retain each backbone's native image preprocessing and representation normalization.

Regression targets and time sampling. We retain each backbone's native prediction parameterization, interpolation convention, time-sampling scheme, and regression loss. WTA uses this regression loss to score auxiliary-code candidates and select the winner. For RAEv2, the candidate loss is the sum of the native main-head and base-head velocity losses, with equal weights.

Post-training configuration. Table 3 summarizes the post-training configurations. We follow each backbone's original training configuration for the global batch size, defined as the total number of training images per optimizer update across devices and gradient-accumulation steps. All four models use auxiliary-code dimension $d = 8$, zero weight decay, and class dropout probability 0.1. SiT uses AdamW with moment coefficients $(0.9, 0.999)$ and a constant learning rate. JiT uses AdamW with $(0.9, 0.95)$ and linearly warms up the learning rate over one epoch before holding it constant. RAE uses AdamW with $(0.9, 0.95)$ and a constant learning rate. RAEv2 retains GMuon, with momentum 0.95 and Nesterov updates, for eligible hidden weight matrices and AdamW with $(0.9, 0.95)$ for embeddings, biases, normalization parameters, output layers, and the auxiliary embedding. The RAEv2 main learning rate is held for the first half epoch and then decays linearly to 10^{-5} at epoch ten. The auxiliary embedding has a $20\times$ learning-rate multiplier and its own 500-update linear warmup, after which it follows the decay of the main schedule.

Table 3: WTA post-training configurations for the four backbones.

Setting	SiT	JiT	RAE	RAEv2
Global batch size	256	1024	1024	1024
Post-training epochs	10	10	10	10
Main learning rate	10^{-4}	2×10^{-4}	2×10^{-4}	5×10^{-5}
EMA decay	0.9999	0.9999	0.9995	0.9995
Gradient-norm limit	—	—	1	1
Random seed	0	0	42	42

A.1.7 Inference and evaluation protocols

For the base-model comparison in Table 2, we follow the sampling protocol used in each model's original paper. Each pretrained generator and its $K = 8$ post-trained counterpart use the same sampling settings, with the post-trained model evaluated at $\lambda = 0$. Each entry uses 50K generated images. The original models' inference-time guidance is disabled. The sampling settings are:

- **SiT:** Euler SDE sampling with 250 steps, a linear diffusion schedule with diffusion norm 1, and a final mean step of size 0.04.

- **JiT:** Heun ODE sampling with 50 steps (99 model evaluations).
- **RAE:** The native Euler ODE configuration with the step-count argument set to 50 and time shift $\sqrt{48}$. The implementation constructs a grid of 50 time points.
- **RAEv2:** Euler ODE sampling with 50 steps and time shift 8. The model's internal guidance scale is set to 1, recovering its unguided prediction.

The SiT entries in Table 2 use 250-step SDE sampling, whereas Table 1 uses the controlled 50-step Euler ODE recipe and reports guidance-selected results.

Metric evaluation. We compute FID, Inception Score, Precision, and Recall using the ADM evaluation suite (Dhariwal & Nichol, 2021). We measure within-class perceptual variation using the original DreamSim ensemble (Fu et al., 2023) and cosine distance between normalized embeddings. We average this distance over all unordered image pairs within each class, and then average the class means with equal class weights. We compute $\mathrm{FD}_{r,6}$ using the released FD-Loss implementation and ImageNet reference statistics of Yang et al. (2026b). Let d_j be the unnormalized Fréchet distance in representation j and a_j its released training–validation reference distance. We report

$$\mathrm{FD}_{r,6} = \frac{1}{6}\sum_{j=1}^{6}\frac{d_j}{a_j}. \tag{14}$$

The six representations are Inception (ADM), ConvNeXt, DINOv2, MAE, SigLIP, and CLIP, with respective normalization constants 1.68, 56.87, 14.19, 0.04, 0.60, and 5.60. We use the released global-feature variants (the `cls` entries for DINOv2, MAE, SigLIP, and CLIP), rather than patch-average variants. Normalization is applied only when forming $\mathrm{FD}_{r,6}$: reported individual FD components retain their raw values. We use the FD-Loss implementation for all six $\mathrm{FD}_{r,6}$ components and the ADM evaluation suite for standalone FID.

A.2 Implementation Details of the One-Dimensional Toy

Source and target distributions. The toy in Fig. 2 transports a standard Gaussian source $\epsilon \sim \mathcal{N}(0,1)$ to a mixture of two compactly supported modes. Let $B \sim \mathrm{Beta}(3,3)$ and let M, independently of B, equal -0.75 with probability 0.25 and $+0.75$ with probability 0.75. We generate target samples as $x_0 = M + 0.6(2B-1)$. Equivalently, the target density is

$$p_{\mathrm{data}}(x) = 0.25\, q_{0.6}(x+0.75) + 0.75\, q_{0.6}(x-0.75), \qquad q_w(s) = \frac{15}{16w}\left(1-\frac{s^2}{w^2}\right)^2 \mathbf{1}_{\{|s|\le w\}}. \tag{15}$$

The two supports are $[-1.35,-0.15]$ and $[0.15,1.35]$, with zero target density in the intervening gap. Training draws fresh source and target samples independently at each update.

Velocity predictor and auxiliary conditioning. The scalar velocity predictor uses three fully connected hidden layers of width 128, each followed by SiLU, and a linear scalar output layer. Its input is the 18-dimensional vector $[x_t, t, \{\sin(j\pi t), \cos(j\pi t)\}_{j=1}^{8}]$. An auxiliary code $z \sim \mathcal{N}(0, I_8)$ passes through a separate $8 \to 128 \to 128$ MLP with SiLU after both linear layers. For each predictor hidden layer, a separate linear map projects this code representation to 256 values, split into a scale $s_\ell(z)$ and a shift $b_\ell(z)$ of dimension 128 each. If a_ℓ is the hidden layer's pre-activation, conditioning gives

$$h_\ell = \mathrm{SiLU}(a_\ell \odot [1 + \lambda s_\ell(z)] + \lambda b_\ell(z))\,. \tag{16}$$

All scale-and-shift projection weights and biases are initialized to zero, so enabling the auxiliary pathway initially preserves the pretrained velocity field.

Flow-matching pretraining. We sample $t \sim \mathcal{U}[0,1]$ and form $x_t = (1-t)\epsilon + t x_0$, with velocity target $u = x_0 - \epsilon$. The predictor is trained with the mean squared error $\mathbb{E}[(v_\theta(x_t,t)-u)^2]$, with auxiliary conditioning disabled. We use Adam with batch size $1{,}024$, default moment coefficients $(0.9, 0.999)$, and gradient-norm clipping at 10. The learning rate is 2×10^{-4} for the first $20{,}000$

updates, 5×10^{-5} for the next $10{,}000$, 10^{-5} for the next $10{,}000$, and 5×10^{-6} thereafter. We train for $50{,}000$ updates and monitor convergence using held-out error against the conditional-mean velocity field computed by numerical quadrature, together with the Wasserstein-1 distance between generated samples and the target distribution. We use the $45{,}000$-update checkpoint, selected based on held-out velocity error, as both the FM baseline and the initialization for WTA post-training.

WTA post-training. For each freshly sampled (x_t, t, u), we draw $K = 8$ independent codes $z_k \sim \mathcal{N}(0, I_8)$ and compute eight predictions using the shared predictor with $\lambda = 1$. We minimize the batch average of $\min_{k\in\{1,\dots,8\}}(v_\theta(x_t, t; c_\phi(z_k)) - u)^2$. Only the minimizing candidate contributes gradients for each training example; both the shared predictor and auxiliary conditioning parameters are updated. Post-training uses a fresh Adam optimizer for $40{,}000$ updates, with learning rate 5×10^{-5} for the first $20{,}000$ updates and 10^{-5} for the remaining $20{,}000$. The batch size, Adam moment coefficients, and gradient clipping are the same as in FM pretraining.

Inference. We generate $8{,}192$ samples per setting from standard Gaussian initial states, reusing the same initial states across FM, auxiliary-on, and auxiliary-off inference. Given the simplicity of this one-dimensional toy, we use a correspondingly short four-step sampling process to highlight differences in off-support generation and mode coverage, illustrating the intuition behind our method. Sampling uses four explicit Euler updates from $t = 0$ to $t = 1$, with $t_i = i/4$ and step size $1/4$:

$$x^{(i+1)} = x^{(i)} + \tfrac{1}{4} v_\theta(x^{(i)}, t_i; c_\phi(z)), \qquad i = 0, 1, 2, 3. \tag{17}$$

Auxiliary-on inference uses $\lambda = 1$ and draws one independent Gaussian code per trajectory, held fixed across all four updates. Auxiliary-off inference uses the same post-trained predictor with $\lambda = 0$, bypassing the code-conditioning pathway. Standard FM sampling uses the pretrained predictor with conditioning disabled. For the trajectory illustration in Fig. 2(c), we use two fixed auxiliary codes at $\lambda = 1$.

A.3 Guidance Search and Evaluation

Constructing guidance curves. We construct guidance curves to compare the overall quality–coverage trends of all methods (the teaser and Fig. 5(c)). We write guided predictions as $v + \gamma(v - v_G)$, where v is the generator prediction (v_λ for our method). For CFG, v_G is the unconditional prediction of the same model. For AG, v_G is the prediction of a weaker guiding model. Within each guidance mechanism, our method and the compared methods use the same grid with spacing 0.25, covering $\gamma \in [0, 4]$ for CFG and $\gamma \in [0, 3]$ for AG.

Our frontier. We explore λ within $[-2.5, 2.5]$ to characterize how auxiliary conditioning and guidance jointly affect quality and coverage. Coarse sweeps show that varying λ shifts the Recall range, while tuning γ primarily improves quality within that range. We use this pattern to construct the SiT frontier in stages rather than exhaustively evaluating the full two-dimensional grid. We first sweep γ at $\lambda = 0$, identifying a favorable interval of $\gamma \in [1.5, 2.0]$. We then vary λ using guidance strengths within this interval. Figure 5(a,b) illustrates how combining guidance sweeps at a few auxiliary-conditioning scales λ already yields an improved quality–coverage frontier. Table 4 lists representative operating points.

Table 4: **Representative 10K operating points for our guided frontier.** Our $K = 8$ model with 50-step Euler sampling.

λ	γ	Recall (%) ↑	$\mathrm{FD}_{r,6}$ ↓
-0.50	1.75	70.73	6.071
-1.50	1.50	75.79	7.992
-1.75	1.50	77.04	8.788
-2.00	1.50	77.98	9.658

Table results. Table 1 reports the best performance of each method in two settings: without a Recall constraint and at a high-Recall target of approximately 67%. For all methods, we select the

configuration with the lowest $\mathrm{FD}_{r,6}$ in each setting, choosing γ from the ranges above on a grid with spacing 0.05. We perform this search coarse-to-fine rather than evaluating every grid point. A coarse sweep at spacing 0.25, with 10K generated samples per configuration, identifies the best-performing γ regions, which we then search at spacing 0.05. We evaluate the selected configurations with 50K images and 50 Euler updates, and Table 5 lists their settings.

Table 5: **Guidance settings for Table 1.** Entries report γ, or (λ, γ) for our $K = 8$ model. For CFG, the conventional scale is $s = 1 + \gamma$. Best settings minimize $\mathrm{FD}_{r,6}$ among evaluated 50K configurations. A dagger denotes a below-target reference.

Model	Guidance	Best $\mathrm{FD}_{r,6}$	Recall $\approx 67\%$
CAFM	CFG	0.90	0.05
REPA	CFG	1.05	0.05
MixFlow	CFG	1.60	$0.00^{\dagger}$
Self-Flow	CFG	0.95	$0.00^{\dagger}$
StableVM	CFG	1.00	0.00
DeepFlow	CFG	1.10	0.00
iREPA	CFG	1.10	$0.00^{\dagger}$
SiT	CFG	1.45	$0.00^{\dagger}$
SiT	AG	2.35	$0.00^{\dagger}$
Ours ($K = 8$)	AG	$(-0.05, 1.80)$	$(-1.25, 1.50)$

B Additional Evaluation and Ablations

B.1 Precision–Recall Evaluation

We also report Precision–Recall curves on ImageNet-256, comparing WTA with recent flow-matching methods across inference-time guidance settings. Precision and Recall characterize sample fidelity and distribution coverage, respectively (Kynkäänniemi et al., 2019). Each operating point is evaluated using 10K generated samples. Figure 10 provides an additional view of the fidelity–coverage trade-off: WTA maintains competitive Precision while extending to higher Recall, highlighting its advantage in high-coverage generation.

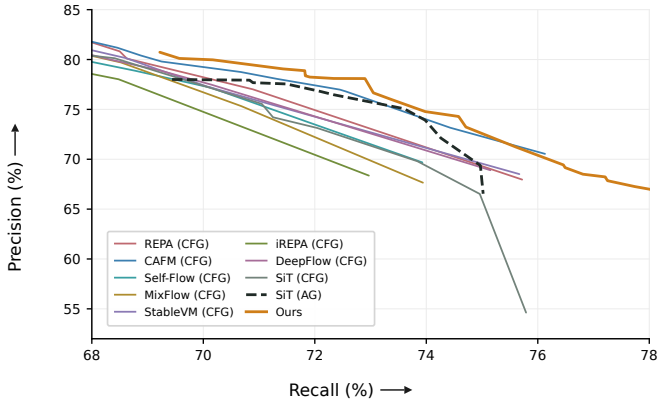


Figure 10: **Precision–Recall comparison in the high-coverage regime.** Results on ImageNet-256 using 10K generated samples per operating point and a 50-step Euler ODE solver. Curves summarize guidance sweeps; WTA also varies the auxiliary-conditioning scale. CFG and AG denote classifier-free guidance and AutoGuidance, respectively. Higher Precision and Recall are better.

B.2 Additional Model Capacity and Continued Training

B.2.1 Are the Gains Due to Additional Model Capacity?

The $K = 1$ control retains the same auxiliary-conditioning architecture and parameter count as our $K = 8$ model, but is post-trained without winner competition. Both models share the pretrained initialization and are compared at matched post-training exposure. This allows us to test whether the gains can be explained by the added model capacity alone.

At $\gamma = 0$, we have already shown that the $K = 1$ model remains close to pretrained SiT at $\lambda = 0$ and responds little to changes in λ (Figs. 8 and 9). We further evaluate this capacity-matched control by varying both λ and γ.

Following the main AutoGuidance setup, we use the $K = 1$ post-trained SiT-XL/2 as the generator and a SiT-B/2 model also post-trained with $K = 1$ as the guide. Figure 11 shows that its guided frontier remains close to the pretrained SiT+AG baseline at high Recall and does not reproduce the high-coverage improvement of our $K = 8$ model.

The generator uses a Gaussian auxiliary code fixed along each trajectory. We sweep its auxiliary-conditioning scale $\lambda \in \{-1.5, -1, -0.5, 0, 0.5, 1, 1.5\}$ and AutoGuidance strength $\gamma \in \{0, 0.25, \ldots, 2\}$, including the native $\lambda = 1$ setting.

Additional check with original SiT-B. We also pair the $K = 1$ post-trained generator with original pretrained SiT-B using the same sweep. This configuration likewise remains close to pretrained SiT+AG at high Recall (Fig. 11). Together with the results at $\gamma = 0$, these controls indicate that added capacity alone does not explain the observed high-coverage improvement.

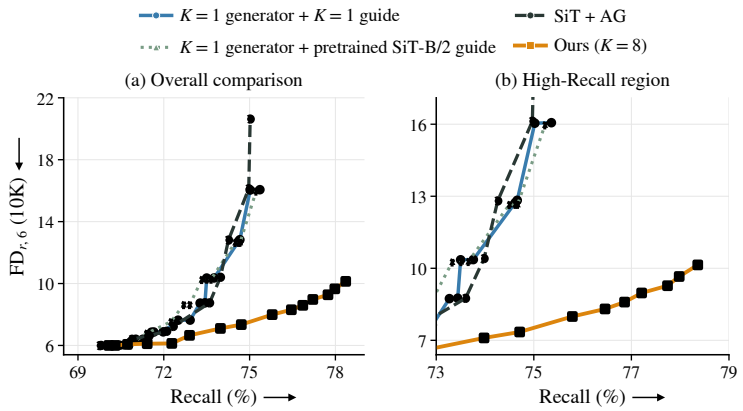


Figure 11: **Additional parameters without competition do not reproduce the high-Recall WTA frontier.** Capacity-matched $K = 1$ controls compared with pretrained SiT+AG and our $K = 8$ model. The $K = 1$ generator is paired with either a $K = 1$ post-trained or an original pretrained SiT-B/2 guide. (a) Overall comparison. (b) High-Recall region. All results use 10K samples and 50-step Euler sampling.

B.2.2 Continued Standard Flow-Matching Training

The $K = 1$ experiment retains the auxiliary architecture while removing competition. We complement it with standard-FM continuation controls, which test additional training without the competitive objective. Table 6 reports SiT-XL/2 continuation results under 10K, 50-step Euler evaluation with $\gamma = 0$. FID remains close to the pretrained value after 50 additional training epochs. These controls therefore do not reproduce the change seen with competitive post-training.

Table 6: **Continued standard FM changes SiT-XL/2 only modestly.** 10K samples, 50-step Euler sampling, $\gamma = 0$.

Model	FID ↓	IS ↑
Pretrained SiT-XL/2	14.41	118.60
FM continuation, 50 epochs	14.31	120.90

Larger training batches. We continue standard FM training of SiT-B/2 for 20 epochs at global batch sizes of 256, 1024, and 8192, starting from the same pretrained initialization (Table 7). Across batch sizes, the overall changes relative to the pretrained model remain limited. Increasing the batch size does not produce a consistent improvement from standard-FM continuation.

Table 7: **Training batch size in standard-FM continuation.** SiT-B/2 after 20 additional data epochs, evaluated using 10K samples, 50-step Euler sampling, and $\gamma = 0$. The pretrained row is the common initialization before continuation.

Model / training batch size	FID ↓	IS ↑
Pretrained SiT-B/2	33.40	51.42
FM continuation, batch 256	32.84	53.45
FM continuation, batch 1024	31.76	56.18
FM continuation, batch 8192	32.81	53.20

B.3 Signs of Auxiliary Conditioning

The main-text analysis shows similar overall responses to positive and negative auxiliary-conditioning scales, with negative scales generally yielding lower $\mathrm{FD}_{r,6}$. We therefore use negative λ values for the guided frontier presented in the main paper. Here, we additionally construct the frontier using positive λ values.

Figure 12 shows that both signs improve the quality–coverage frontier over pretrained SiT with AutoGuidance in the high-coverage region. Empirically, negative scales generally provide a slightly better trade-off.

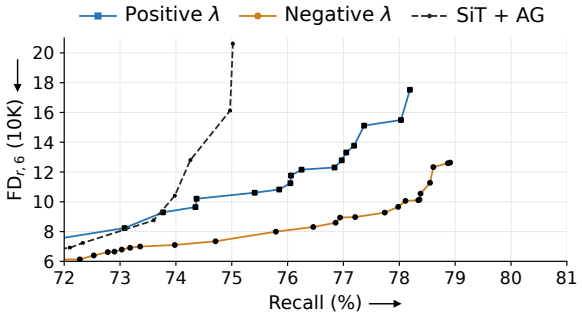


Figure 12: **Both signs of auxiliary conditioning improve the high-coverage frontier.** $K = 8$ guided frontiers for positive and negative λ, compared with pretrained SiT AutoGuidance baseline. Both signs improve the frontier in the high-coverage region. All results use 10K samples and 50-step Euler sampling.

B.4 Effect of Competition Level K

B.4.1 SiT at $\gamma = 0$

Figure 13 complements Fig. 9 with $\mathrm{FD}_{r,6}$, FID, and Inception Score as the auxiliary-conditioning scale λ varies with $\gamma = 0$. We use the same SiT-XL/2 checkpoint for each K throughout these comparisons, with $K \in \{1, 4, 8, 16\}$.

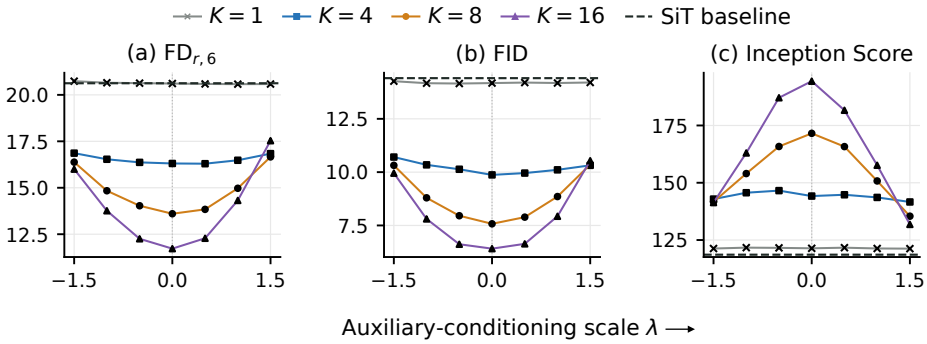


Figure 13: **Competition changes the response to auxiliary conditioning.** (a) $\mathrm{FD}_{r,6}$, (b) FID, and (c) Inception Score for $K \in \{1, 4, 8, 16\}$ with $\gamma = 0$. The dashed horizontal lines denote pretrained SiT. All curves use the same λ values and 10K samples with 50-step Euler sampling.

The $K = 1$ curves remain nearly flat and close to pretrained SiT. With competition, the post-trained base models at $\lambda = 0$ attain lower $\mathrm{FD}_{r,6}$ and FID, and higher Inception Score. Varying λ exposes a broader response for $K = 8$ and $K = 16$ than for $K = 4$. Together with the Precision–Recall curves,

these results show that competition changes both the base model and the behaviors accessible through auxiliary conditioning.

B.4.2 GUIDED FRONTIERS ON SIT

We next combine auxiliary conditioning with AutoGuidance. For each fixed λ, varying the inference-time guidance strength γ traces a quality–coverage trajectory. Each generator is paired with a SiT-B/2 guiding model post-trained with the same K.

Figure 14 shows that guided-frontier improvements persist across $K = 4, 8, 16$. Compared with pretrained SiT+AG, all three improve the quality–coverage trade-off in the overlapping high-coverage region, with larger K extending the attained Recall range.

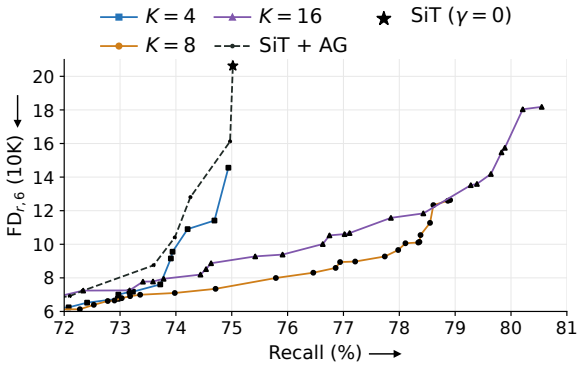


Figure 14: **The high-coverage frontier improvement persists across competition levels.** Measured guided frontiers for $K = 4, 8, 16$, with pretrained SiT AutoGuidance and its $\gamma = 0$ point as references. The plot shows the high-coverage region. All results use 10K samples and 50-step Euler sampling.

B.4.3 GUIDED FRONTIERS ON RAEV2

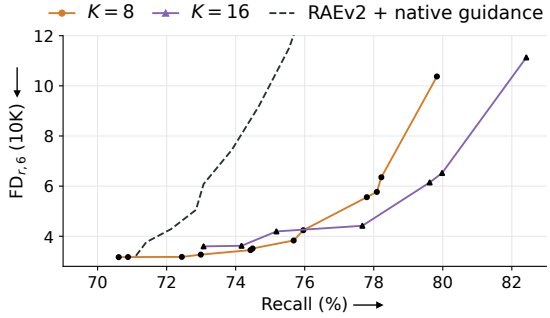


Figure 15: **The RAEv2 frontier improvement persists across competition levels.** Measured $\mathrm{FD}_{r,6}$–Recall frontiers for $K = 8, 16$ and pretrained RAEv2 with its native guidance. Each setting uses 10K samples and native Euler-50 sampling with timestep shift 8. Two pretrained-reference points have $\mathrm{FD}_{r,6}$ values above the plotted range.

We extend the $K = 8$ comparison in Figure 7 by including $K = 16$. For both competition levels, we sweep the auxiliary-conditioning scale λ and the RAEv2-guidance strength γ, using RAEv2's native sampler (Appendix A.1.7) and guidance window $t \in [0.1, 1.0]$. Both competition levels improve the quality–coverage frontier over pretrained RAEv2, and $K = 16$ extends it further into the high-coverage region (Figure 15).

B.5 AUXILIARY-CODE SAMPLING POLICIES

We compare the policies introduced in Section 3.2: *fixed-code sampling* holds one Gaussian auxiliary code throughout each trajectory, while *resampled-code sampling* draws a new code at every Euler update. Both use the same SiT-XL/2 WTA model ($K = 8$), with 10K images per setting, 50 Euler updates, and matched initial noises and class labels. At $\lambda = 0$, both recover the same post-trained base model.

Varying the auxiliary-conditioning scale ($\gamma = 0$). Figure 16 shows similar overall responses to auxiliary-conditioning strength under both sampling policies, with fixed-code sampling generally yielding lower $\mathrm{FD}_{r,6}$ than resampled-code sampling.

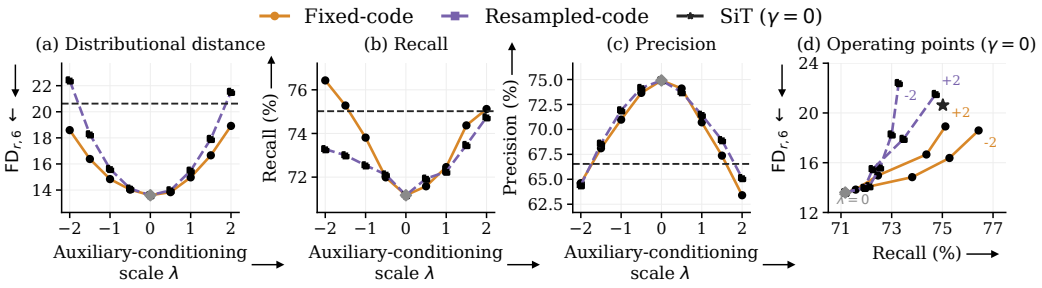


Figure 16: **Varying auxiliary conditioning under two sampling policies ($\gamma = 0$).** SiT-XL/2 WTA ($K = 8$), 10K images per setting, 50 Euler updates. Panels (a–c) show $\mathrm{FD}_{r,6}$, Recall, and Precision versus auxiliary-conditioning scale; dashed black horizontal lines show pretrained SiT at $\gamma = 0$. Panel (d) shows the same observations in Recall–$\mathrm{FD}_{r,6}$ space, with a black star for SiT. Gray diamonds denote the shared post-trained base model ($\lambda = 0$). In (d), filled and open policy markers denote negative and positive scales, respectively; endpoint labels give λ.

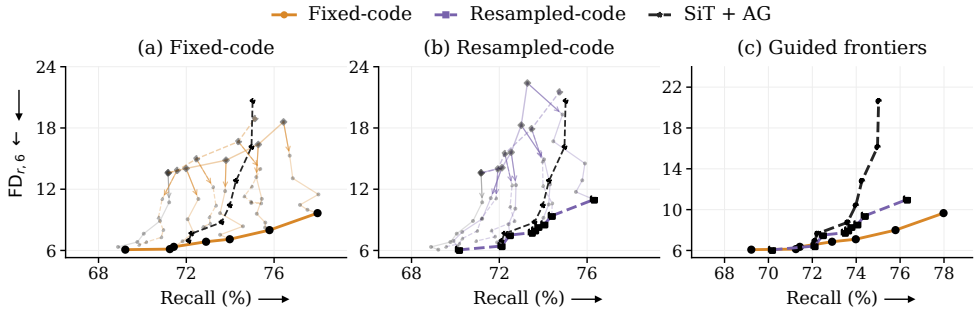


Figure 17: **Both code policies improve the guided frontier, with a larger broad-coverage gain from fixed codes.** All results use 10K samples and 50-step Euler sampling. In (a,b), pale diamonds connect the $\gamma = 0$ operating points; pale paths vary γ at fixed λ, with arrows pointing toward increasing guidance. Solid and dashed pale paths denote nonpositive and positive λ, respectively; gray denotes the post-trained base model ($\lambda = 0$). (c) compares these frontiers directly with original SiT+AG, using the same axis ranges.

Guided frontiers. We compare both sampling policies using the same (λ, γ) settings. As shown in Figure 17, resampled-code sampling also improves the quality–coverage frontier over SiT+AG, while fixed-code sampling yields a stronger frontier, particularly at high recall.

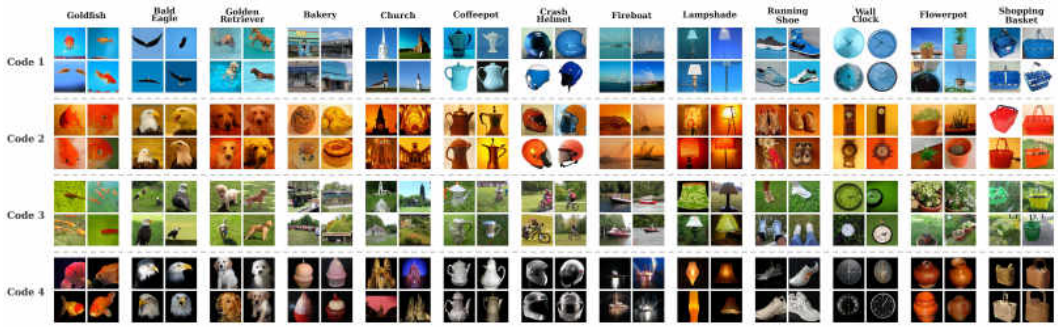
Interpretation. Holding the auxiliary code fixed may allow its influence to persist across sampling steps, whereas resampling changes that influence at each step. This offers a possible explanation for the stronger high-coverage frontier observed with fixed-code sampling.

B.6 Reusable Auxiliary-Code Signatures

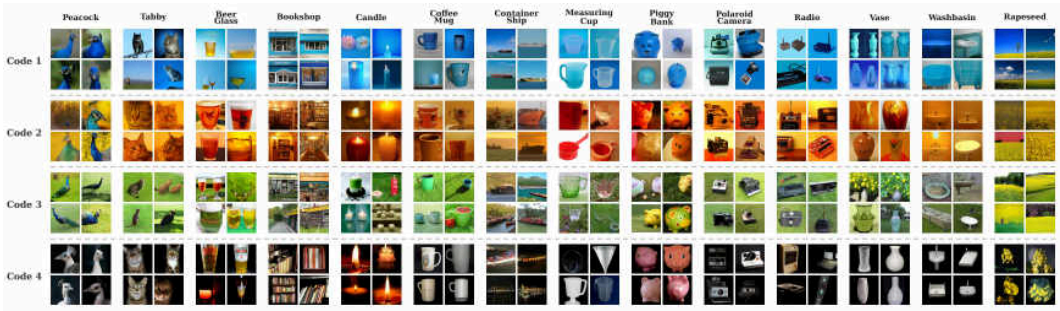
To further investigate the effect of holding an auxiliary code fixed, we analyze how individual codes shape generated samples across initial noises and classes.

The experiments in this section use fixed-code sampling with $\lambda = 1$, the training-time auxiliary-conditioning scale, and $\gamma = 0$.

Although auxiliary codes are sampled independently during training and are not assigned persistent roles, samples generated with the same fixed code exhibit recurring appearance patterns across initial noises and classes (Figure 18).



(a) Reusable visual signatures across ImageNet classes.



(b) Additional classes with the same auxiliary codes.

Figure 18: **Samples generated with the same auxiliary code share recurring appearance patterns across source noises and classes.** Each row fixes one auxiliary code z and each column one ImageNet class. The four samples at each row–column intersection vary only the initial noise ϵ. Both panels use the same SiT-XL/2 WTA checkpoint, the same four auxiliary codes, and 50-step ODE sampling at $\lambda = 1$, $\gamma = 0$, holding the code fixed throughout each trajectory. Repeated tendencies in color, background, and composition appear both within and across classes.

B.6.1 Code Signatures Strengthen with Competition

We quantify how consistently these code-specific patterns recur across classes and how this changes with K. Specifically, we test whether the auxiliary code can be identified from generated samples on held-out classes. We fix 64 auxiliary codes for analysis and split 128 ImageNet classes into a reference half and a held-out half.

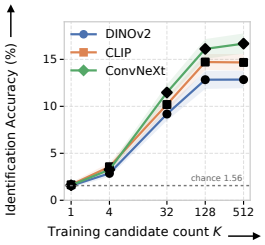


Figure 19: **Code signatures strengthen with competition.** Identification accuracy on held-out classes, over 64 fixed analysis codes; chance is 1.56%. The matched K sweep uses 128 classes and four source-noise seeds, with 50-step ODE sampling and fixed auxiliary codes at $\lambda = 1$, $\gamma = 0$. Shaded bands are 95% hierarchical class–seed bootstrap intervals.

We first define the change a code makes: for a given class and source noise, we generate one sample with the code and one without, and compute the difference of their features in a pretrained visual encoder's space (e.g., DINOv2). We then subtract the average of this difference over all codes, leaving only what is specific to this code. Averaging these code-specific differences over the reference classes gives the code's signature. For each held-out sample, we compute its code-specific difference in the same way, and assign it to the code with the nearest signature. Chance is $1/64$, and identification succeeds only if a code's effect recurs on classes never used to build its signature.

Across DINOv2, CLIP, and ConvNeXt features, identification accuracy is near chance for the $K = 1$ control and rises with competition over the measured range, reaching roughly ten times chance at $K = 512$ (Figure 19).

B.7 Guidance Configurations

B.7.1 AutoGuidance Configuration

We compare alternative guiding models and auxiliary-conditioning settings. The configuration used in our main experiments achieves the strongest high-coverage frontier among those evaluated (Fig. 20).

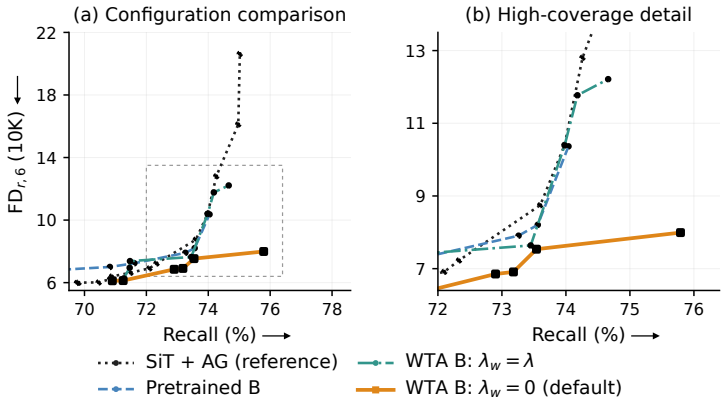


Figure 20: **AutoGuidance guiding-model configurations.** The three configurations share the same WTA-post-trained $K = 8$ generator and the same (λ, γ) settings; only the guiding model or its auxiliary conditioning changes. The default uses WTA-post-trained SiT-B/2 with $\lambda_w = 0$. (a) Configuration comparison. (b) Detail of the boxed high-coverage region. SiT+AG uses the reference guidance sweep. All results use 10K samples and 50-step Euler sampling.

Compared configurations. We hold the WTA-post-trained SiT-XL/2 generator fixed ($K = 8$) and compare three guiding-model configurations: the original pretrained SiT-B/2; a WTA-post-trained SiT-B/2 ($K = 8$) with auxiliary conditioning enabled; and the same post-trained guiding model with auxiliary conditioning disabled, as used in the main experiments. For the enabled configuration, both models receive the same numerical auxiliary code and conditioning scale, $\lambda_w = \lambda$. For the default configuration, the guiding model uses $\lambda_w = 0$.

Evaluation. All three configurations use the same generator conditioning scales $\lambda \in \{-1.5, -1, -0.5, 0.5, 1, 1.5\}$ and inference-time guidance strengths $\gamma \in \{0.5, 1, 1.5, 2\}$, follow-

ing Eq. 7. Each setting uses 10K samples with matched initial noises and 50-step Euler sampling. A Gaussian auxiliary code is held fixed along each sampling trajectory. Pretrained SiT-XL/2 guided by pretrained SiT-B/2 provides the SiT+AG reference.

B.7.2 Combining with Classifier-Free Guidance

We also combine auxiliary conditioning with classifier-free guidance (CFG) and compare our post-trained SiT-XL/2 model with $K = 8$ against pretrained SiT-XL/2 with CFG. The guiding prediction is the class-unconditional output of the same model. The conditional and unconditional predictions use the same auxiliary code and conditioning scale λ, with the code fixed along each trajectory. Figure 21 shows that our post-trained model improves the quality–coverage trade-off over pretrained SiT with CFG in the displayed Recall range.

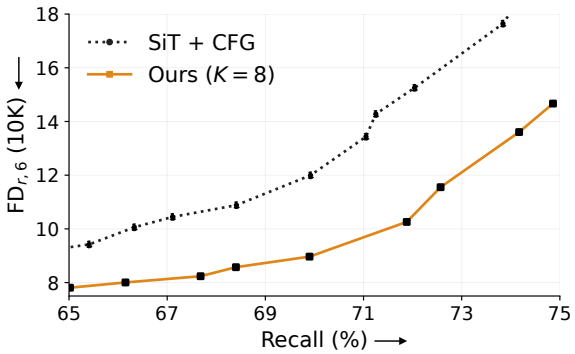


Figure 21: Quality–coverage tradeoffs with classifier-free guidance. Pretrained SiT+CFG is compared with our post-trained model using $K = 8$. Our curve is constructed from a joint (λ, γ) sweep using one fixed trained model. All results use 10K images and 50-step Euler sampling.

C Qualitative Results

C.1 Qualitative Improvements in the Base Model

Our post-training improves the fidelity of the base model itself, even without auxiliary conditioning or additional guidance ($\lambda = 0$, $\gamma = 0$). The SiT and JiT examples in Figures 22 and 23 illustrate these improvements.

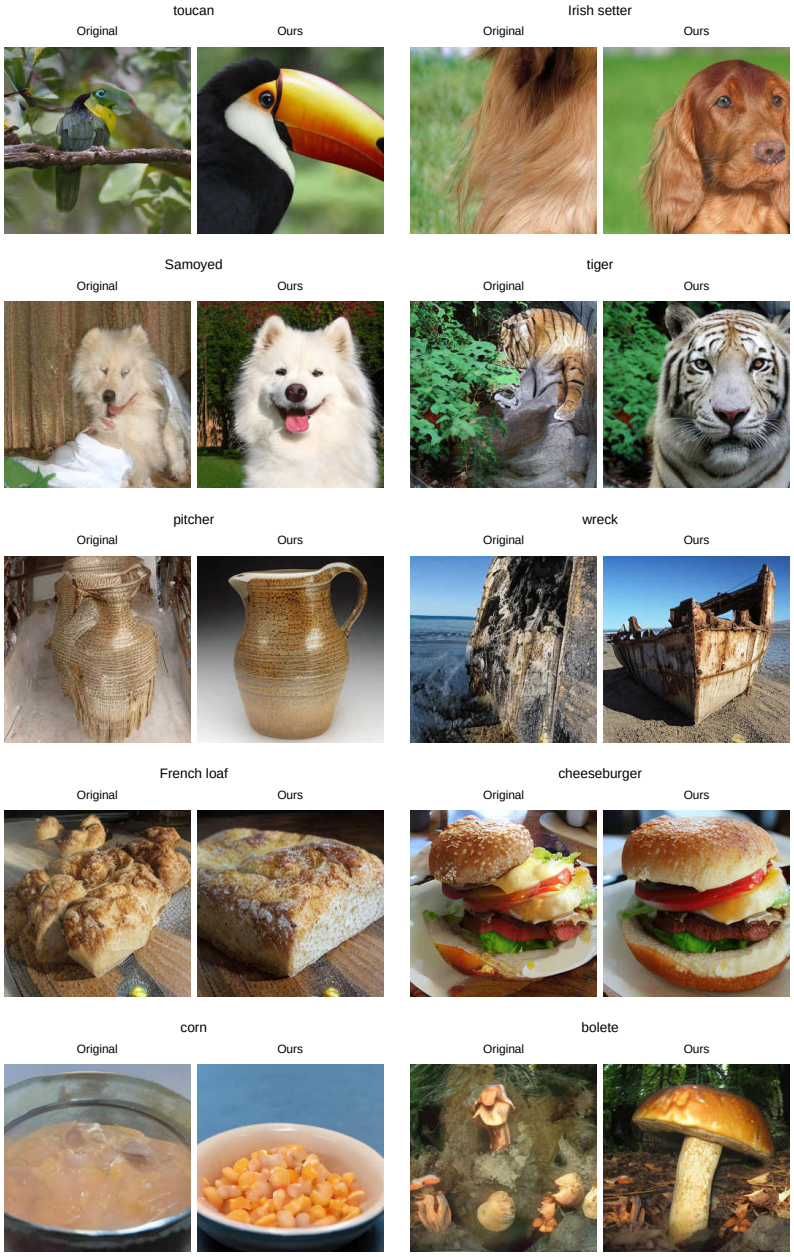


Figure 22: **Base-model improvements on SiT-XL/2.** Ten pairs of ImageNet-256 samples from the original pretrained model (left) and our post-trained model (right). Each pair uses the same class and initial noise. Both models use 50 Euler steps without additional guidance. Our model uses $\lambda = 0$ and $\gamma = 0$.

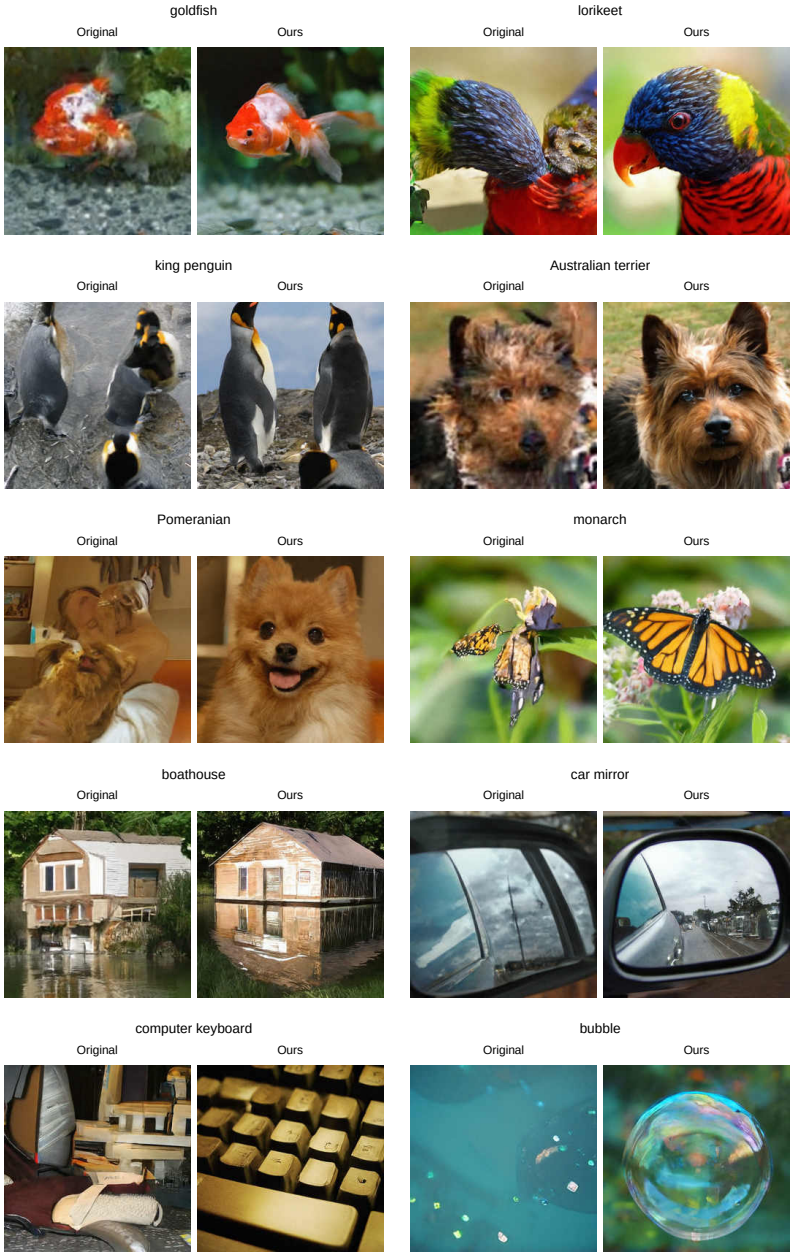


Figure 23: **Base-model improvements on JiT-H/16.** Ten pairs of ImageNet-256 samples from the original pretrained model (left) and our post-trained model (right). Each pair uses the same class and initial noise. Both models use 50-step Heun sampling without additional guidance. Our model uses $\lambda = 0$ and $\gamma = 0$.

C.2 Structured Variation Under Auxiliary Conditioning and Guidance

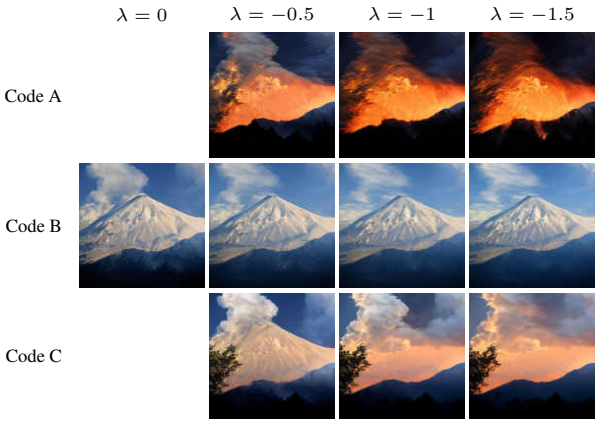
To examine how auxiliary conditioning and guidance change individual samples, we visualize samples generated with different auxiliary codes across auxiliary-conditioning scales and guidance strengths.

For each example, we fix the ImageNet class and initial noise and randomly sample three auxiliary codes, each held fixed throughout sampling. Panel (a) shows one code per row and $\lambda \in \{0, -0.5, -1, -1.5\}$ across columns at $\gamma = 0$. Panel (b) repeats this grid at AG strengths $\gamma = 0, 0.75, 1.5$, showing the shared $\lambda = 0$ image once per block. All images are generated by SiT-XL/2 with $K = 8$ and 50-step Euler sampling.

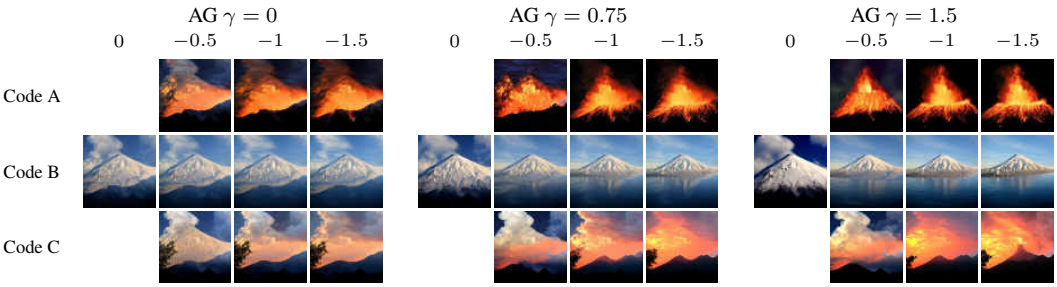
At $\lambda = 0$ and $\gamma = 0$, sampling yields the post-trained base-model image. As $|\lambda|$ increases, samples from different codes often share the visual structure of this image while developing increasingly distinct appearance and details. Increasing the AG strength γ can further accentuate these differences and sharpen image details.

Volcano

(a) Auxiliary conditioning ($\gamma = 0$)

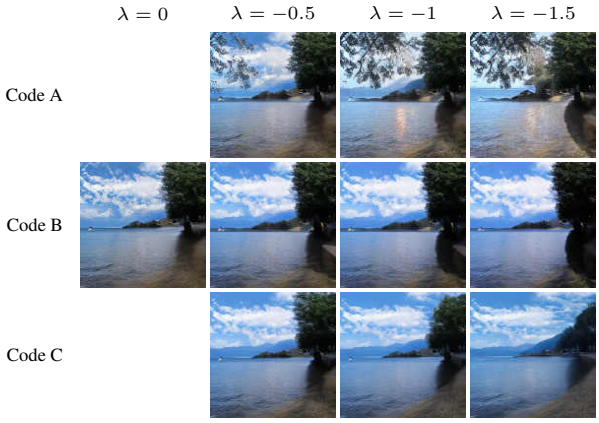


(b) Guidance across auxiliary scales

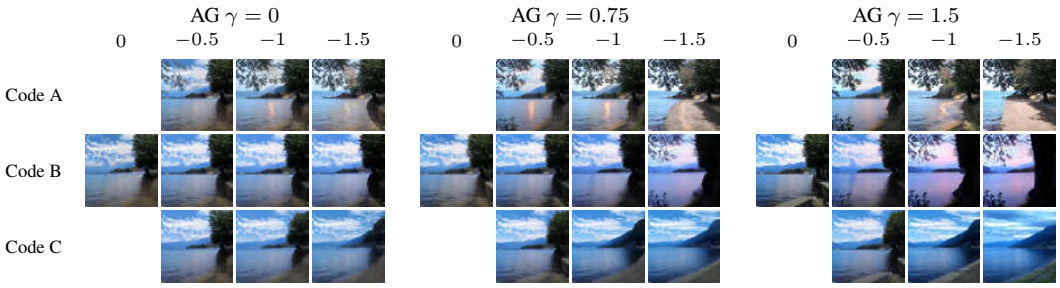


Lakeside

(a) Auxiliary conditioning ($\gamma = 0$)

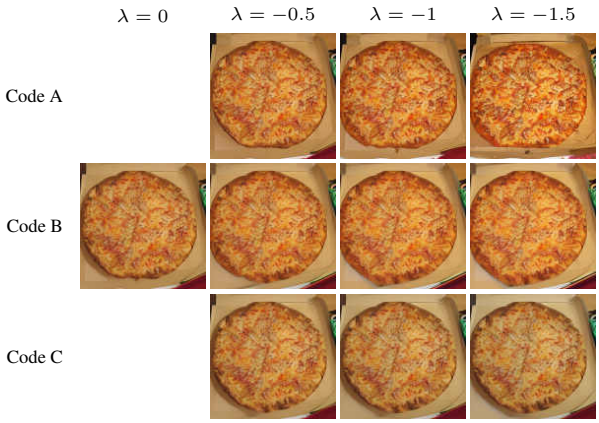


(b) Guidance across auxiliary scales

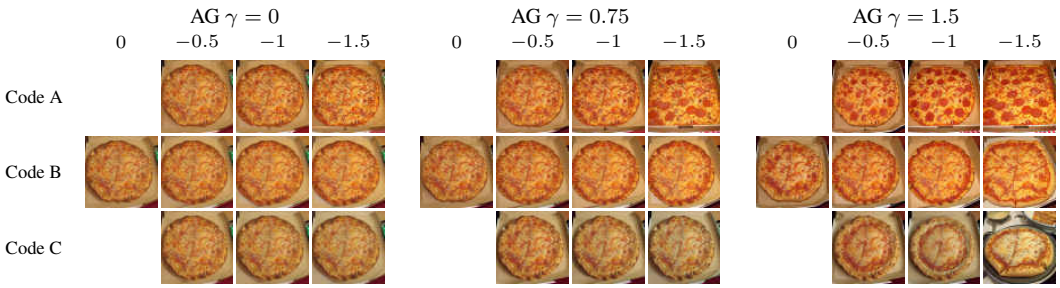


Pizza

(a) Auxiliary conditioning ($\gamma = 0$)

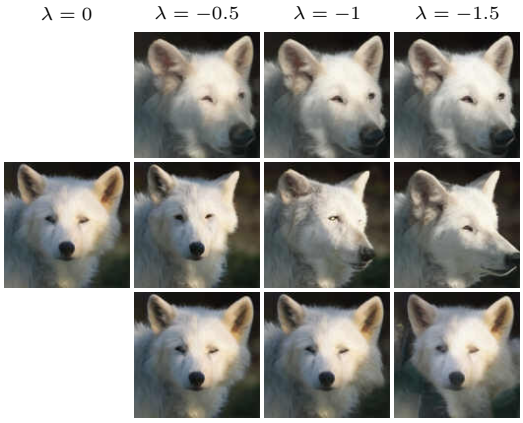


(b) Guidance across auxiliary scales

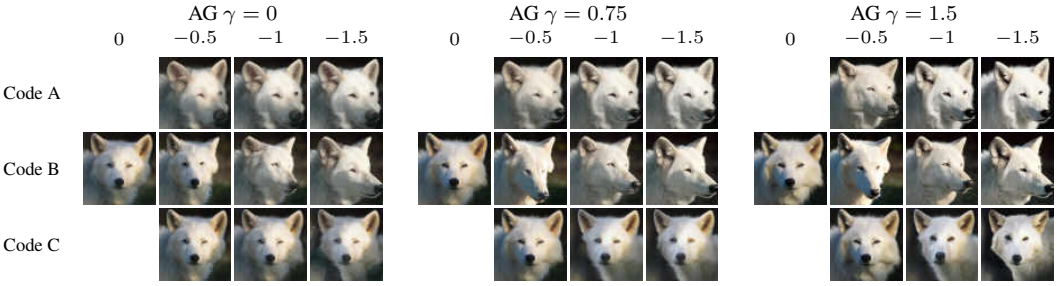


White wolf

(a) Auxiliary conditioning ($\gamma = 0$)

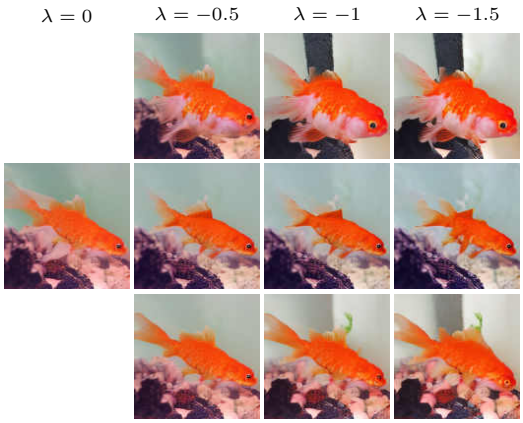


(b) Guidance across auxiliary scales

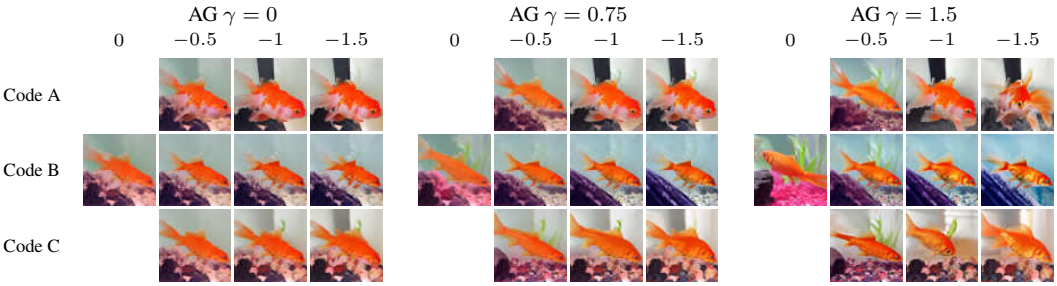


Goldfish

(a) Auxiliary conditioning ($\gamma = 0$)

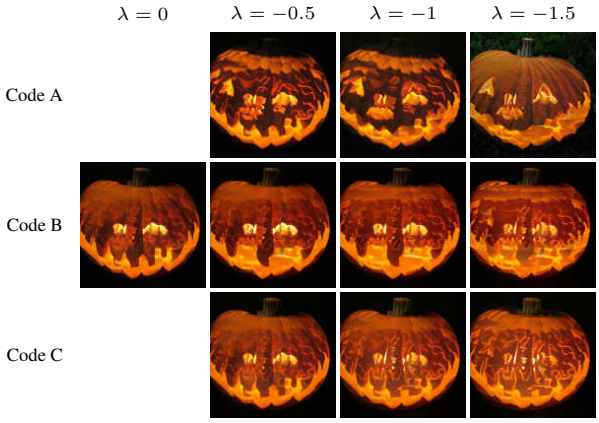


(b) Guidance across auxiliary scales

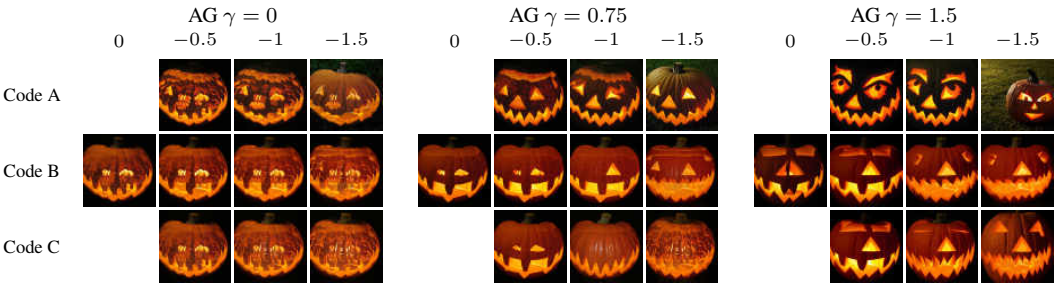


Jack-o'-lantern

(a) Auxiliary conditioning ($\gamma = 0$)

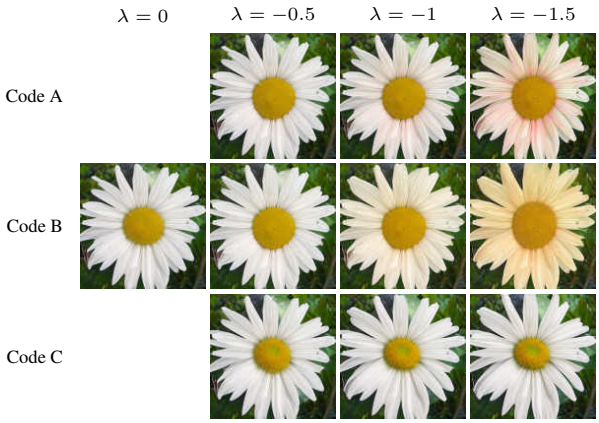


(b) Guidance across auxiliary scales

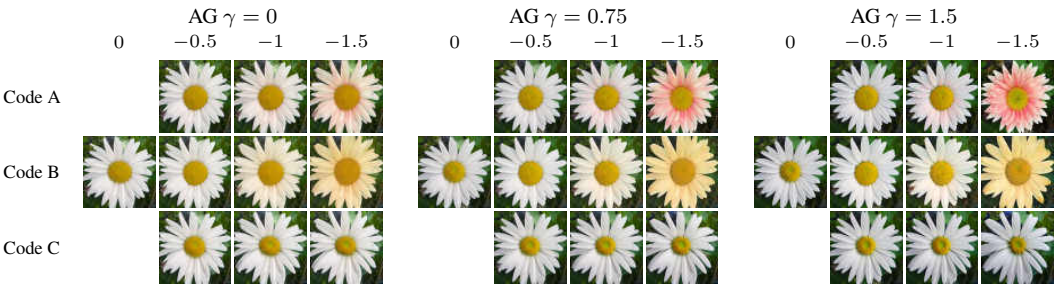


Daisy

(a) Auxiliary conditioning ($\gamma = 0$)

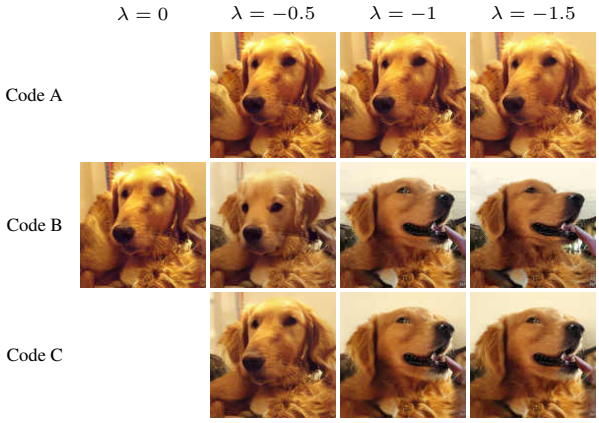


(b) Guidance across auxiliary scales

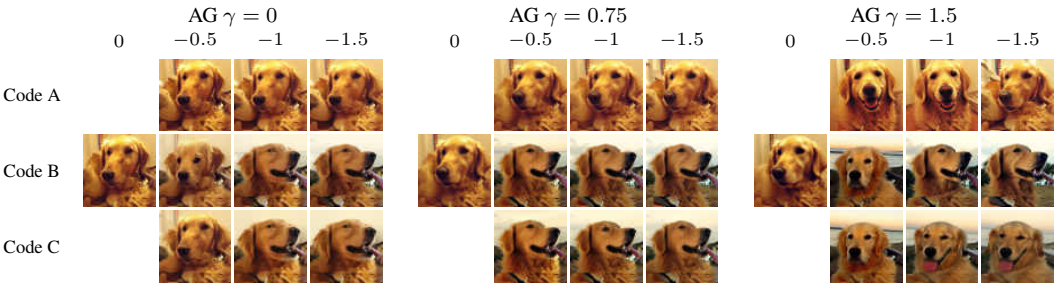


Golden retriever

(a) Auxiliary conditioning ($\gamma = 0$)

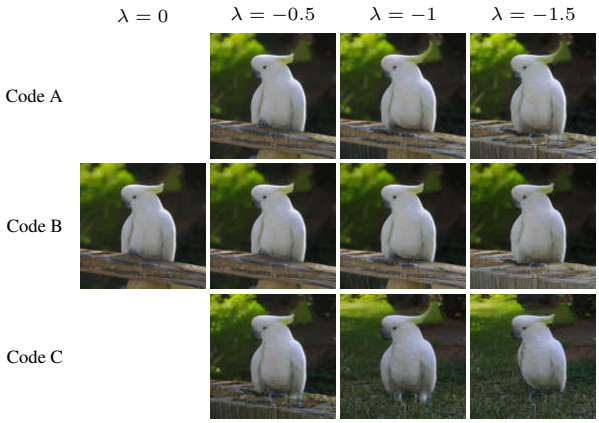


(b) Guidance across auxiliary scales

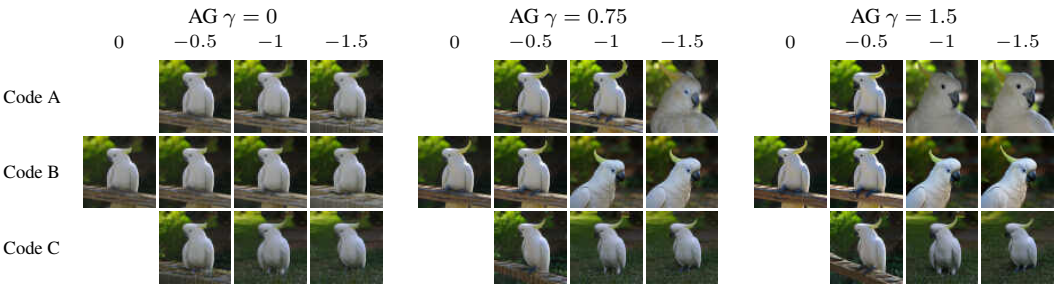


Sulphur-crested cockatoo

(a) Auxiliary conditioning ($\gamma = 0$)



(b) Guidance across auxiliary scales



We also extend these visualizations to both positive and negative auxiliary-conditioning scales, complementing the quantitative comparison in Appendix B.3.

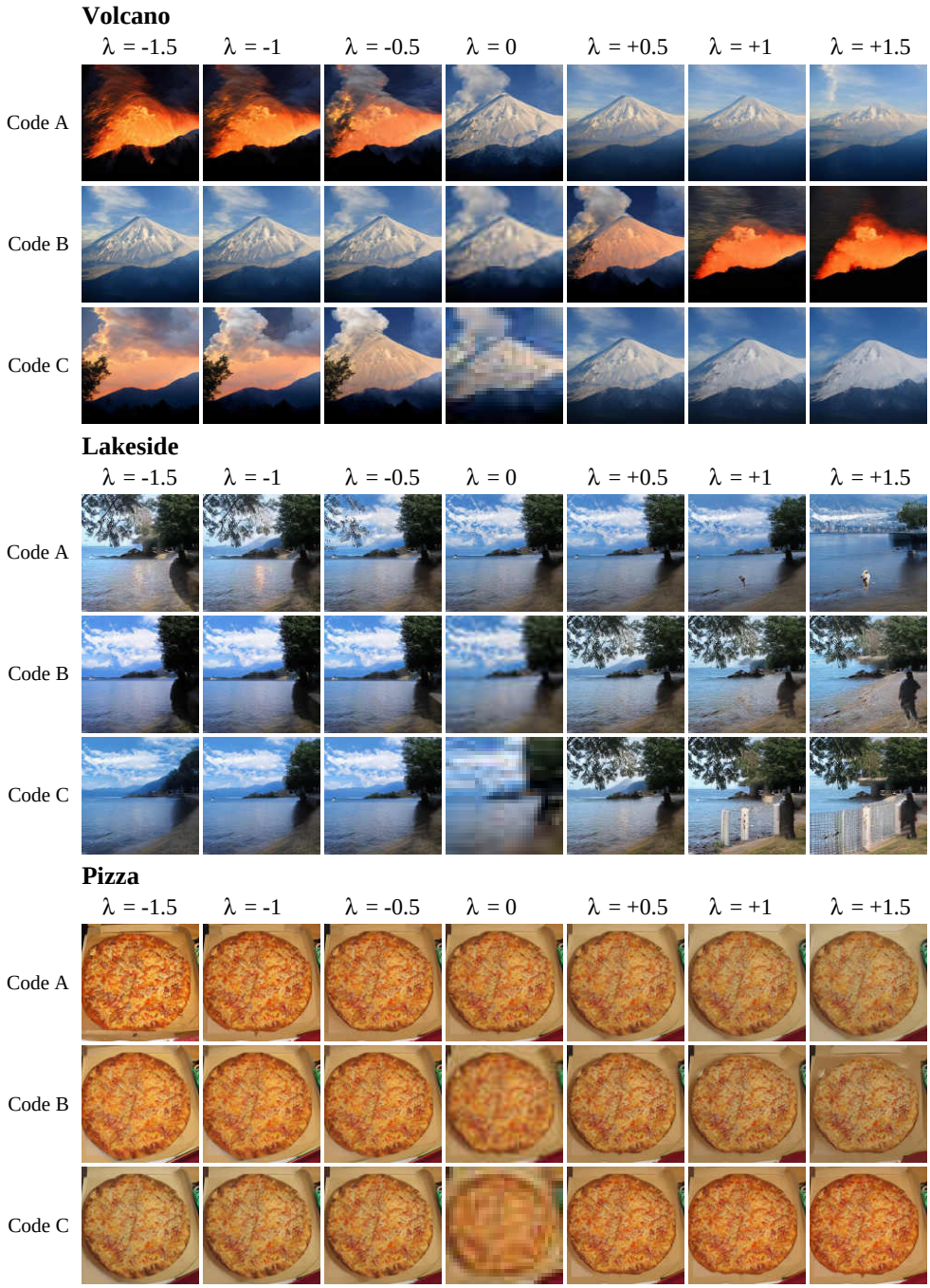


Figure 24: **Positive and negative auxiliary-conditioning scales.** Columns show $\lambda \in \{-1.5, -1, -0.5, 0, 0.5, 1, 1.5\}$. Within each class, the initial noise is fixed; within each row, the auxiliary code is fixed. All samples use $\gamma = 0$, SiT-XL/2 $K = 8$, and 50-step Euler sampling.

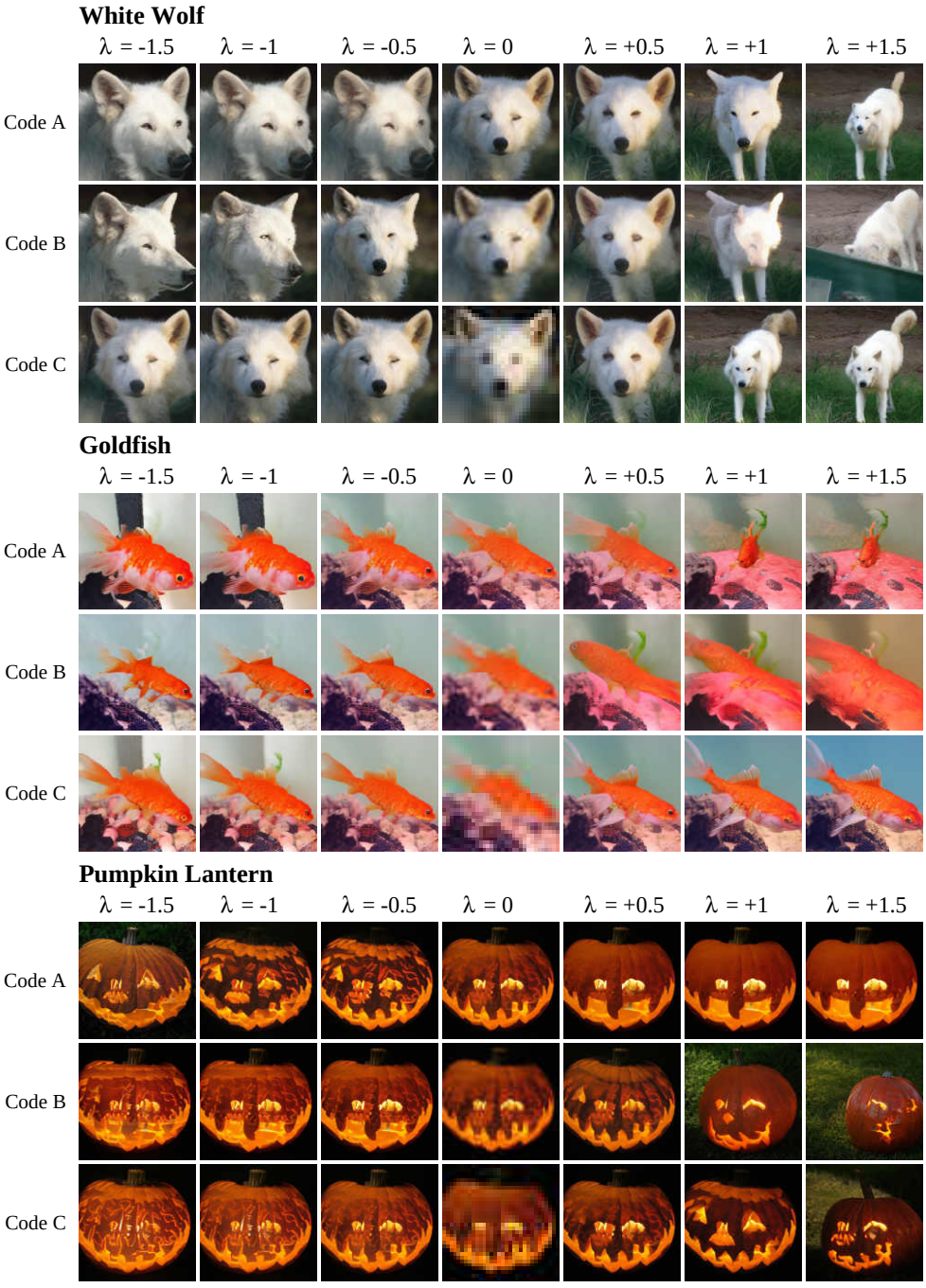


Figure 25: **Positive and negative auxiliary-conditioning scales on additional classes.** The layout and sampling settings follow Figure 24.

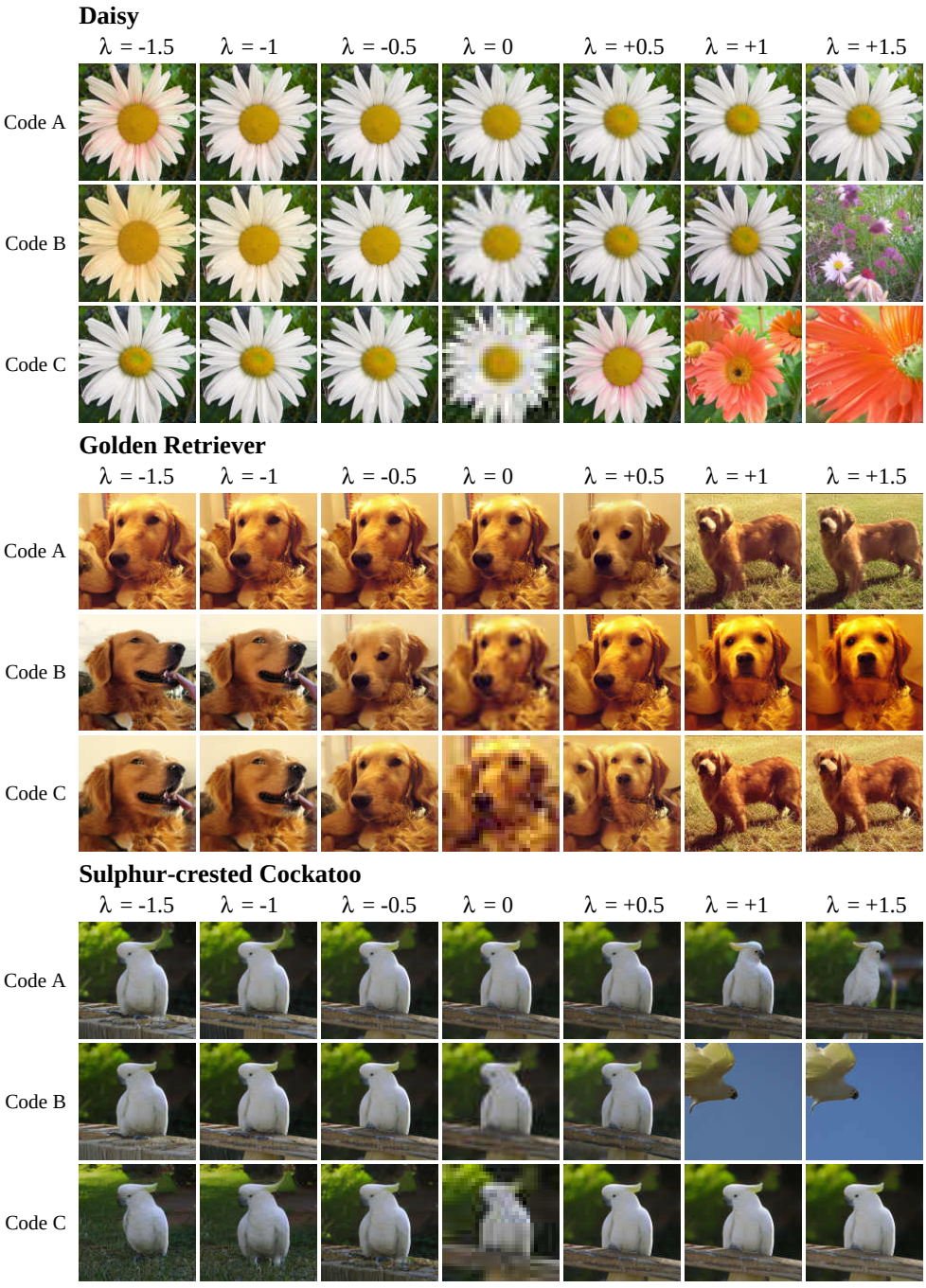


Figure 26: **Positive and negative auxiliary-conditioning scales on additional classes.** The layout and sampling settings follow Figure 24.

D COMPUTATIONAL COST

D.1 TRAINING COST AND SCALING WITH THE NUMBER OF CANDIDATES

We report the training budget, model-core computation, and measured training throughput and GPU memory usage of WTA post-training for SiT-XL/2.

D.1.1 TRAINING BUDGET AND MODEL-CORE COMPUTATION

For each training example, we evaluate K candidate predictions without retaining their gradient graphs, select the lowest-loss candidate, and recompute its prediction with gradients before backpropagation. Candidate scoring is performed in chunks, where the chunk size specifies the number of candidate–example pairs processed together. Smaller chunks limit activation memory, while larger chunks can improve throughput through more efficient batching. Chunking changes how candidate evaluations are grouped, without changing their total number.

Let F_0 and F_1 denote the forward multiply–accumulate (MAC) counts per example for ordinary SiT-XL/2 and its auxiliary-conditioned counterpart. At global batch size B, using two FLOPs per MAC and a backward-to-forward computation ratio β, the model-core FLOPs per update are

$$C_{\mathrm{FM}} = 2B(1+\beta)F_0, \qquad C_{\mathrm{ours}}(K) = 2B(K+1+\beta)F_1, \tag{18}$$

$$R(K) = \frac{C_{\mathrm{ours}}(K)}{C_{\mathrm{FM}}} = \frac{K+1+\beta}{1+\beta}\frac{F_1}{F_0}. \tag{19}$$

Following the analytical convention used by DiT (Peebles & Xie, 2023), we use $\beta = 2$. Counting MACs in the SiT-XL/2 implementation gives $F_0 = 118.621$ GMAC and $F_1 = 118.848$ GMAC per example. The count includes the auxiliary-conditioning projections and excludes VAE encoding, elementwise operations, optimizer operations, and communication. At $K = 8$, the model-core computation per update is approximately $3.674\times$ that of ordinary FM.

Ten-epoch training budget. At global batch size 256, ten epochs correspond to 50,040 optimizer updates. The original SiT-XL/2 pretraining schedule uses seven million updates at the same batch size (Ma et al., 2024). Post-training uses 0.715% of this update budget. Accounting for candidate computation gives

$$\frac{C_{\mathrm{post}}}{C_{\mathrm{pre}}} = \frac{50{,}040}{7{,}000{,}000}\,R(8) \approx 0.0263. \tag{20}$$

Thus ten-epoch post-training at $K = 8$ requires approximately **2.63% of the original pretraining model-core computation**. Table 8 summarizes the training budget.

Table 8: Training budget for SiT-XL/2. The pretraining schedule follows Ma et al. (2024). Model-core computation uses the analytical FLOP accounting in Equation (19).

Quantity	Original pretraining	$K = 8$ post-training
Training epochs	—	10
Optimizer updates	7,000,000	50,040
Global batch size	256	256
Example presentations	1,792,000,000	12,810,240
Relative update budget	100%	0.715%
Relative model-core computation	100%	2.63%

D.1.2 MEASURED TRAINING EFFICIENCY

We measure training throughput on eight NVIDIA H100 80GB GPUs with global batch size 256, using FP32 with TF32 enabled. For each K, we select the highest-throughput configuration from the tested microbatch, accumulation, and candidate-chunk settings, subject to a peak reserved-memory limit of 70 GiB per GPU. Ordinary FM uses the same microbatch choices. All reported configurations

use a per-GPU microbatch of 32 without gradient accumulation. At $K = 1$, we retain auxiliary conditioning and use the same candidate evaluation and winner recomputation procedure.

We repeat each measurement twice, with 50 warmup updates followed by three blocks of 50 timed updates per run. We report the median throughput across the six blocks. We time complete training updates, including data loading and transfer, online VAE encoding, forward and backward computation, optimizer updates, and EMA. Checkpoint saving and preview generation are excluded. Throughput is aggregated across all eight GPUs using the slowest rank's elapsed time, and peak reserved memory is the maximum across ranks.

Table 9: Training throughput and peak reserved GPU memory on eight H100 80GB GPUs, using the highest-throughput tested configuration for each method. Throughput is reported across all eight GPUs, and memory per GPU.

Method	Candidate chunk	Images/s ↑	GiB/GPU
Ordinary FM	—	834.51	36.55
Ours, $K = 1$	32	656.86	39.66
Ours, $K = 2$	64	581.01	39.67
Ours, $K = 4$	128	474.36	39.66
Ours, $K = 8$	256	343.75	39.66
Ours, $K = 16$	512	224.12	40.96
Ours, $K = 32$	512	129.79	40.93

Training throughput decreases as K increases, while chunked candidate evaluation keeps peak reserved memory within approximately 41 GiB per GPU across the reported configurations (Table 9). Inference does not require evaluating all K candidates. The number of model evaluations per sampling step depends on the guidance configuration.